

WINDOWS SERVER: DOMINE A
FORÇA JEDI DO SISTEMA

WELCOME TO FORTINET



8

LISTA DE FIGURAS

Figura 1 – Disquetes de 3 ½ polegadas (Pergunte a seus pais)	6
Figura 2 – Sistemas operacionais, embelezando coisas há décadas	8
Figura 3 – Gerenciador de janelas GNOME.....	9
Figura 4 – Tom's Window Manager (twm).....	9
Figura 5 – Veja essa imagem. Aposto que lembrou do clássico “bramir do elefante”	12
Figura 6 – Porque, como diriam os guitarristas, valvulado é mais legal	14
Figura 7 – Sistema em lote (batch) antigo na segunda geração	16
Figura 8 – Estrutura de uma tarefa FMS	16
Figura 9 – Partições da Memória	21
Figura 10 – Jovem Steve Jobs ilustrando uma parede do museu da Apple em Praga	28
Figura 11 – Discos flexíveis 3 ½ polegadas necessários para instalar o Windows 95	29
Figura 12 – Um antigo e saudoso Palm Top, caso você nunca tenha visto um	33
Figura 13 – Máquina de lavar roupa também tem sistema embarcado, sabia?	34
Figura 14 – Placas de prototipação Arduino, ideais para sensores modernos.....	35
Figura 14 – Já imaginou um sistema de UTI dando tela azul da morte? Não pode, né?	36
Figura 16 – Funcionamento das <i>threads</i>	40
Figura 17 – Dinâmica dos processos	43
Figura 18 – Estados de um processo	53
Figura 19 – Comportamento de um processo	57
Figura 20 – Três modos simples de organizar a memória	67
Figura 21 – Ilustração do problema de realocação	69
Figura 22 – A posição e a função da MMU	75
Figura 23 – Entrada típica de uma tabela de páginas	76

LISTA DE QUADROS

Quadro 1 – Alguns objetivos do algoritmo de escalonamento sob diferentes circunstâncias.....	62
--	----

EMENDAS

SUMÁRIO

1 INTRODUÇÃO	5
2 O SISTEMA OPERACIONAL ABSTRAINDO COMPLEXIDADE	5
3 SISTEMA OPERACIONAL COMO UM GERENCIADOR DE RECURSOS	10
4 HISTÓRIA DA COMPUTAÇÃO (E DOS SISTEMAS OPERACIONAIS)	12
4.1 O primeiro “computador”	12
4.2 A primeira geração (1945-1955) - válvulas.....	13
4.3 A segunda geração (1955-1965) – transistores e sistemas em lote (batch)	15
4.4 A terceira geração (1965-1980) - CIs e multiprogramação.....	17
4.5 A quarta geração (1980-presente) - computadores pessoais.....	25
5 TIPOS DE SISTEMAS OPERACIONAIS	30
5.1 Sistemas operacionais de computadores de grande porte.....	30
5.2 Sistemas operacionais de servidores.....	31
5.3 Sistemas operacionais de multiprocessadores	32
5.4 Sistemas operacionais de computadores pessoais.....	32
5.5 Sistemas operacionais de computadores portáteis	33
5.6 Sistemas operacionais embarcados.....	34
5.7 Sistemas operacionais de nós sensores (<i>sensor node</i>)	34
5.8 Sistemas operacionais de tempo real	35
5.9 Sistemas operacionais de cartões inteligentes (<i>smart cards</i>).....	37
6 THREADS	37
6.1 O uso da Thread	38
7 PROCESSO	41
7.1 O modelo de processo	42
7.2 Criação de processos.....	45
7.3 Término de processos.....	49
7.4 Hierarquias de processos.....	50
7.5 Estados de processos	51
8 ESCALONAMENTO	54
8.1 Comportamento do processo	57
8.2 Quando escalonar	58
8.3 Categorias de algoritmos de escalonamento	60
8.4 Objetivos do algoritmo de escalonamento.....	61
9 GERENCIAMENTO DE MEMÓRIA.....	65
9.1 Sem abstração de memória	66
9.2 Executando múltiplos programas sem abstração de memória.....	68
9.3 Noção de endereçamento	72
10 MEMÓRIA VIRTUAL	73
10.1 Paginação	74
10.2 Tabela de páginas.....	75
10.3 Estrutura de uma entrada da tabela de páginas.....	76
CONCLUSÃO.....	78
REFERÊNCIAS.....	79

1 INTRODUÇÃO

Sistema Operacional é algo um tanto complexo de se definir. Falar que é um software que executa em modo núcleo ou no núcleo do computador seria uma definição superficial. Parte do problema ocorre porque os sistemas operacionais realizam, basicamente, duas funções não relacionadas: fornecer aos programadores de aplicativos (e aos programas aplicativos, naturalmente) um conjunto de recursos abstratos claros, em vez de recursos confusos de *hardware*, e gerenciar esses recursos de hardware, otimizando ao máximo os recursos físicos. Dependendo do tipo de usuário, ele vai lidar mais com uma função ou com outra.

Trata-se de um verdadeiro “maestro” da sintonia computacional ou, se preferir, a alma (*software*) de um corpo feito de silício (*hardware*).

2 O SISTEMA OPERACIONAL ABSTRAINDO COMPLEXIDADE

A arquitetura (conjunto de instruções, organização de memória, E/S e estrutura de barramento) da maioria dos computadores em níveis de linguagem de máquina é primitiva e de difícil programação, especialmente na entrada/saída. Para tornar isso mais concreto, examinemos rapidamente como é feita a E/S da unidade de discos flexíveis (disquetes) a partir de um chip controlador compatível com o NEC-PD765.

Esse chip é usado na maioria dos computadores pessoais baseados em processadores Intel. Usamos discos flexíveis como exemplo, porque, embora sejam obsoletos, são muito mais simples que os discos rígidos modernos. O PD765 tem 16 comandos, especificados pela carga de 1 a 9 bytes no registrador do dispositivo.

Esses comandos são para leitura e escrita de dados, movimentação do braço do disco e formatação de trilhas. Servem também para inicialização, sinalização, reinicialização e recalibração dos controladores e das unidades de disquetes.



Figura 1 – Disquetes de 3 ½ polegadas (Pergunte a seus pais)
Fonte: Shutterstock (2019)

Os comandos mais básicos são `read` e `write`, cada um deles requer 13 parâmetros agrupados em 9 bytes. Esses parâmetros especificam itens como os endereços dos blocos de dados a serem lidos, o número de setores por trilha, o modo de gravação usado no meio físico, o espaço livre entre setores e o que fazer com um marcador de endereço de dados removidos.

Se essas palavras não fizeram sentido para você, não se preocupe: é assim mesmo, um tanto místico. Quando a operação é completada, o chip controlador retorna 23 campos de status e de erros agrupados com 7 bytes. Como se isso não bastasse, o programador de unidades de discos flexíveis ainda deve saber se o motor está ligado ou não. Se estiver desligado, ele deverá ser ligado (com um longo atraso de inicialização) antes que os dados possam ser lidos ou escritos.

O motor não pode permanecer ligado por muito tempo, senão o disco flexível poderá sofrer desgaste. O programador, então, será forçado a equilibrar dois fatores: longos atrasos de inicialização versus desgastes do disco flexível (e a perda dos dados nele gravados). Dúvida cruel!

Sem entrar em detalhes, de fato, é claro que um programador de nível médio provavelmente não se envolverá profundamente com os detalhes de programação das unidades de discos flexíveis (ou discos rígidos, que são piores de se fazer). Em vez disso, o programador busca lidar com essas unidades de um modo mais abstrato e simples.

No caso dos discos, uma abstração típica seria aquela compreendida por um disco que contém uma coleção de arquivos com nomes. Cada arquivo pode ser aberto para leitura ou escrita e, então, ser lido ou escrito e, por fim, fechado. Detalhes como: se a gravação deveria usar uma modulação por frequência modificada e qual seria o estado atual do motor não apareceriam na abstração apresentada ao programador da aplicação.

Abstração é o elemento-chave para gerenciar complexidade. Boas abstrações transformam uma tarefa quase impossível em duas manejáveis. A primeira delas é definir e implementar as abstrações. A segunda seria usar essas abstrações para resolver o problema à mão.

Uma abstração que quase todo usuário de computação entende é o arquivo. Ele é um fragmento de informação útil, como uma foto digital, uma mensagem de e-mail salva ou uma página da Web. Lidar com fotos, e-mails e páginas da Web é mais fácil do que manipular detalhes de discos, como o disco flexível descrito anteriormente.

A tarefa do sistema operacional é criar boas abstrações e, em seguida, implementar e gerenciar os objetos abstratos criados. Elas são um dos elementos cruciais para compreender os sistemas operacionais.

Esse ponto é tão importante que convém repeti-lo em outras palavras. Com todo o respeito devido aos engenheiros industriais que projetaram o Macintosh, o *hardware* é feio. Processadores reais, memórias, discos e outros dispositivos são muito complicados e apresentam interfaces difíceis, desajeitadas e incoerentes para que as pessoas que precisam escrever *softwares* as utilizem.

Algumas vezes, isso se deve à necessidade de compatibilidade com a versão anterior do *hardware*, outras vezes, ao desejo de economizar dinheiro, mas ainda há vezes em que os projetistas de *hardware* não percebem os problemas que estão causando ao *software* (ou não se importam com tais problemas). Pelo contrário, uma das principais tarefas do sistema operacional é ocultar o *hardware* e oferecer aos programas (e seus programadores) abstrações precisas, claras, elegantes e coerentes com as quais trabalhar. Os sistemas operacionais transformam o feio em bonito.



Figura 2 – Sistemas operacionais, embelezando coisas há décadas
Fonte: Shutterstock (2019)

Deve-se observar que os clientes reais do sistema operacional são os programas aplicativos (via programadores de aplicativos, naturalmente). São eles que lidam diretamente com o sistema operacional e suas abstrações. Por outro lado, os usuários finais lidam com abstrações fornecidas pela interface do usuário, seja a linha de comandos *shell* ou uma interface gráfica.

Embora as abstrações da interface com o usuário possam ser semelhantes às fornecidas pelo sistema operacional, nem sempre é o caso. Para esclarecer esse ponto, considere a área de trabalho normal do Windows e o *prompt* de comando, orientado a linhas de comando (a famosa tela escura). Ambos são programas executados no sistema operacional e usam as abstrações que o Windows fornece, mas oferecem interfaces de usuário muito diferentes.

De modo semelhante, um usuário Linux executando um gerenciador de janelas Gnome ou KDE (vide Figura “Gerenciador de janelas GNOME”) vê uma interface muito diferente daquela vista por um usuário trabalhando diretamente sobre o X-Window System, em um gerenciador mais simples (como o twm, vide Figura “Tom’s Window Manager (twm)”), mas as abstrações do sistema operacional subjacente são as mesmas em ambos os casos.

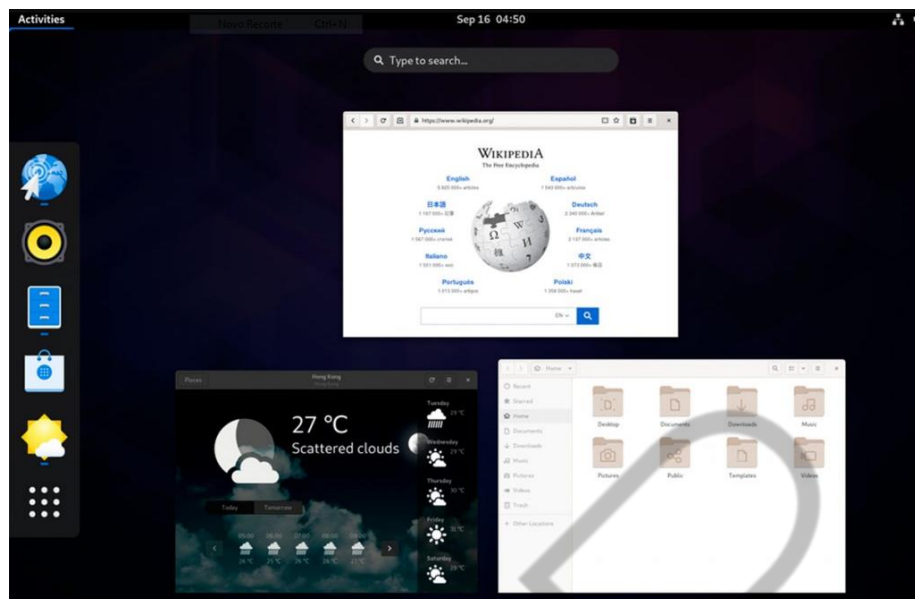


Figura 3 – Gerenciador de janelas GNOME
Fonte: Wikimedia Commons (2007)

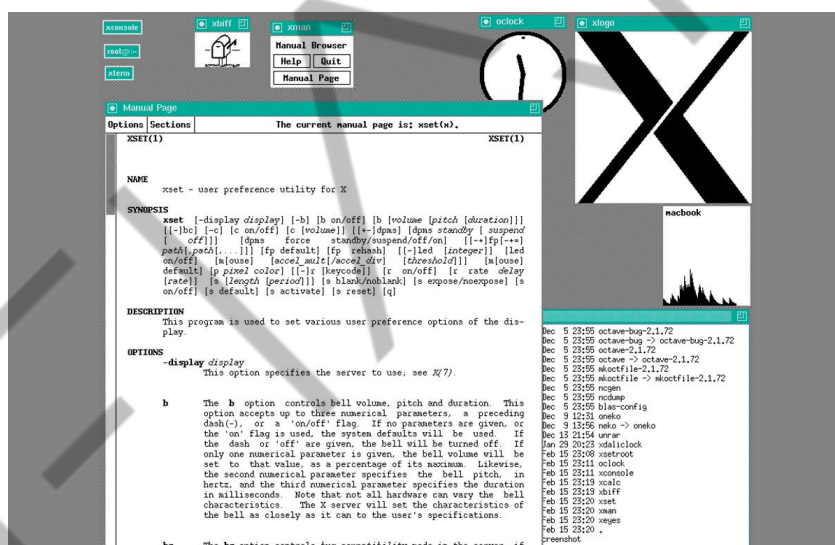


Figura 4 – Tom's Window Manager (twm)
Fonte: Wikimedia Commons (2007)

Neste capítulo, estudaremos as abstrações fornecidas pelos programas aplicativos em mais detalhes, mas falaremos muito menos sobre interfaces com o usuário. Esse é um tema amplo e importante, mas apenas periféricamente relacionado com sistemas operacionais.

3 SISTEMA OPERACIONAL COMO UM GERENCIADOR DE RECURSOS

O conceito de um sistema operacional como provedor de uma interface conveniente a seus usuários é uma visão *top-down* (abstração do todo para as partes). Em uma visão alternativa, *bottom-up* (abstração das partes para o todo), o sistema operacional gerencia todas as partes de um sistema complexo.

Computadores modernos são constituídos de processadores, memórias, temporizadores, discos, dispositivos apontadores tipo mouse, interfaces de rede, impressoras e uma ampla variedade de outros dispositivos (dê uma conferida em nosso capítulo anterior!). Segundo essa visão, o trabalho do sistema operacional é fornecer uma alocação ordenada e controlada de processadores, memórias e dispositivos de E/S entre vários programas que competem por eles.

Sistemas operacionais modernos permitem que múltiplos programas sejam executados ao mesmo tempo. Imagine o que aconteceria se três programas em execução em algum computador tentassem imprimir suas saídas simultaneamente, na mesma impressora. As primeiras linhas poderiam ser do programa 1, as linhas seguintes seriam do programa 2, algumas outras do programa 3, e assim por diante. Ou seja, uma tremenda confusão, como diria o narrador da *Sessão da Tarde*. O sistema operacional pode trazer paz e ordem a essa confusão em potencial, armazenando temporariamente, no disco, todas as saídas destinadas à impressora, formando uma fila (de impressão), dizendo quem pode o que e quando.

O sistema operacional poderia então enviar sua saída, que estaria no arquivo em disco, para a impressora. Ao mesmo tempo, o outro programa poderia continuar gerando mais saídas, que não estariam, obviamente, indo para a impressora (ainda).

Quando um computador (ou uma rede) tem múltiplos usuários, a necessidade de gerenciar e proteger a memória, dispositivos de E/S e outros recursos é muito maior, já que, de outra maneira, os usuários poderiam interferir uns nos outros. Além disso, os usuários, muitas vezes, precisam compartilhar não somente *hardware*, mas também informação (arquivos, bancos de dados etc.).

Em resumo, essa visão do sistema operacional mostra que sua tarefa principal é manter o controle sobre quem está usando qual recurso, garantindo suas requisições de recursos, controlando as contas e mediando conflitos de requisições entre diferentes programas e usuários.

O gerenciamento de recursos realiza o compartilhamento (ou multiplexação) desses recursos de duas maneiras diferentes: no tempo e no espaço. Quando um recurso é compartilhado (multiplexado) no tempo, diferentes programas ou usuários aguardam sua vez de usá-lo. Primeiro, um deles obtém o uso do recurso, daí outro, e assim por diante.

Por exemplo, com somente uma CPU e múltiplos programas precisando ser executados nela, o sistema operacional aloca a CPU a um programa, e após ele ser executado por tempo suficiente, outro programa obtém seu uso, então outro e, por fim, o primeiro programa novamente.

Determinar como o recurso é compartilhado no tempo, quem vai depois de quem e por quanto tempo, é tarefa do sistema operacional. Outro exemplo de compartilhamento no tempo se dá com a impressora. Quando múltiplas saídas são enfileiradas para imprimir em uma única impressora, deve-se decidir sobre qual será a próxima saída a ser impressa.

O outro tipo de compartilhamento (multiplexação) é o de espaço. Em lugar dos consumidores esperando sua vez, cada um ocupa uma parte do recurso. Por exemplo, a memória principal é normalmente dividida entre vários programas em execução (ou processos). Assim, cada um pode residir ao mesmo tempo na memória (por exemplo, a fim de ocupar a CPU temporariamente).

Existindo memória suficiente para abrigar múltiplos programas, é mais eficiente mantê-los nela em vez de destinar toda a memória a um só deles, especialmente se o programa precisar apenas de uma pequena fração do total. Naturalmente, isso levanta questões sobre justiça, proteção etc., e cabe ao sistema operacional resolvê-las.

Outro recurso que é compartilhado no espaço é o disco (rígido). Em vários sistemas, um único disco pode conter arquivos de muitos usuários ao mesmo tempo. Alocar espaço em disco e manter o controle sobre quem está usando quais parcelas do disco é uma típica tarefa de gerenciamento de recursos do sistema operacional. Ufa!

4 HISTÓRIA DA COMPUTAÇÃO (E DOS SISTEMAS OPERACIONAIS)

Para compreendemos um pouco melhor como os computadores e, por consequência, os sistemas operacionais funcionam, vale fazer uma viagem na história dessas tecnologias. Entre em sua TARDIS e viagem conosco!



Figura 5 – Veja essa imagem. Aposto que lembrou do clássico “bramir do elefante”
Fonte: Shutterstock (2019)

4.1 O primeiro “computador”

Os sistemas operacionais vêm passando por um processo gradual de evolução. Nas próximas seções veremos algumas das principais fases dessa evolução. Como a história dos sistemas operacionais é bastante ligada à arquitetura de computadores sobre a qual eles são executados, veremos as sucessivas gerações de computadores para entendermos as primeiras versões de sistemas operacionais.

Esse mapeamento das gerações de sistemas operacionais em relação a gerações de computadores é um tanto vago, mas revela a existência de certa estrutura.

A sequência de eventos apresentada a seguir é, em grande medida, cronológica, mas foi um percurso acidentado. Os desenvolvimentos não esperaram que os anteriores terminassem antes de se iniciarem. Houve muitas sobreposições, para não falar de muitos falsos começos e becos sem saída. Considere-a como um guia, não como a palavra final.

O primeiro computador digital foi projetado pelo matemático inglês Charles Babbage (1792-1871). Embora Babbage tenha empregado a maior parte de sua vida e de sua fortuna para construir sua 'máquina analítica', ele nunca conseguiu vê-la funcionando de modo apropriado, pois era inteiramente mecânica e a tecnologia da sua época não poderia produzir as rodas, as engrenagens e as correias de alta precisão que eram necessárias. É óbvio que a máquina analítica não possuía um sistema operacional.

Outro aspecto histórico interessante foi que Babbage percebeu que seria preciso um software para sua máquina analítica. Para isso, ele contratou uma jovem chamada Ada Lovelace, filha do famoso poeta inglês Lord Byron, como a primeira programadora do mundo. A linguagem de programação Ada® foi assim denominada em sua homenagem.

4.2 A primeira geração (1945-1955) - válvulas

Depois dos infrutíferos esforços de Babbage, seguiram-se poucos progressos na construção de computadores digitais até a Segunda Guerra Mundial, que estimulou uma explosão de atividades. O professor John Atanasoff e seu então aluno de graduação Clifford Berry construíram o que consideramos o primeiro computador digital com funcionamento, na Universidade do Estado de Iowa. Ele usava 300 válvulas.

Quase ao mesmo tempo, Konrad Zuse, em Berlim, construiu o computador Z3, de relês. Em 1944, o Colossus foi desenvolvido por um grupo em Betchley Park, Inglaterra, o Mark foi construído por Howard Aiken, em Harvard, e o ENIAC foi construído por William Mauchley e seu aluno de graduação J. Presper Eckert, na Universidade da Pensilvânia. Alguns eram binários, alguns usavam válvulas, alguns eram programáveis, mas todos eram muito primitivos e levavam longos segundos para executar até os cálculos mais simples.

Naquela época, um mesmo grupo de pessoas projetava, construía, programava, operava e realizava a manutenção de cada máquina. Toda a programação era feita em código de máquina absoluto e muitas vezes conectando *plugs* em painéis para controlar as funções básicas da máquina. Não havia linguagens de programação (nem mesmo a linguagem Assembly existia), e os sistemas operacionais também ainda não haviam sido inventados.

O modo normal de operação era o seguinte: o programador reservava, antecipadamente, tempo de máquina em uma planilha, ia para a sala da máquina, inseria seu painel de programação no computador e passava algumas horas torcendo para que nenhuma das 20 mil válvulas queimasse durante a execução. Praticamente todos os problemas eram cálculos numéricos diretos, como determinar tabelas de senos, cossenos e logaritmos.

No início da década de 1950, essa rotina havia sido aprimorada com a introdução das perfuradoras de cartões. Era possível, então, escrever programas com cartões e lê-los em lugar de painéis de programação; de outra maneira, o procedimento seria o mesmo.



Figura 6 – Porque, como diriam os guitarristas, valvulado é mais legal
Fonte: Shutterstock (2019)

4.3 A segunda geração (1955-1965) – transistores e sistemas em lote (batch)

A introdução do transistor em meados da década de 1950 mudou o quadro radicalmente: os computadores se tornaram suficientemente confiáveis para que pudessem ser fabricados e comercializados com a expectativa de que continuariam a funcionar por tempo suficiente para executar algum trabalho útil. Pela primeira vez, havia uma clara separação entre projetistas, fabricantes, programadores e técnicos da manutenção.

Essas máquinas, então denominadas computadores de grande porte (*mainframes*), ficavam isoladas em salas especiais com ar-condicionado, operadas por equipes profissionais. Somente grandes corporações, agências governamentais ou universidades podiam pagar vários milhões de dólares para tê-las.

Para uma tarefa (isto é, um programa ou um conjunto de programas) ser executada, o programador primeiro escrevia o programa no papel (em Fortran ou em linguagem Assembly) e depois o perfurava em cartões. Ele levava o maço de cartões para a sala de entradas, entregava-o a um dos operadores e ia tomar um café até que a saída impressa estivesse pronta mais tarde.

Ele, então, apanhava um dos maços de cartões que haviam sido trazidos para a sala de entradas e o colocava na leitora de cartões. Se fosse necessário um compilador Fortran, o operador precisava retirar do armário o maço de cartões correspondente e lê-lo. Muito tempo de computador era desperdiçado enquanto os operadores andavam pela sala das máquinas.

Por causa do alto custo do equipamento, era natural que se começasse a buscar maneiras de reduzir o desperdício de tempo no uso da máquina. A solução geralmente adotada era a do sistema em lote (ou também chamado de *batch*). A ideia era gravar várias tarefas em fita magnética, usando um computador relativamente mais barato, como o IBM 1401, que era muito bom para ler cartões, copiar fitas e imprimir saídas, mas não tão eficiente em cálculos numéricos. Outras máquinas mais caras, como a IBM 7094, eram usadas para a computação propriamente dita.

Depois de aproximadamente uma hora acumulando um lote de tarefas, os cartões eram lidos em uma fita magnética, que era encaminhada para a sala das máquinas, onde era montada em uma unidade de fila. O operador, então, carregava um programa especial (o antecessor do sistema operacional de hoje), que lia a primeira tarefa da fita e executava-a. A saída não era impressa, mas gravada em uma segunda fita. Depois de cada tarefa terminada, o sistema operacional automaticamente lia a próxima tarefa da fita e começava a executá-la.

Quando todo o lote era processado, o operador retirava as fitas de entrada e de saída, trocava a fita de entrada com a do próximo lote e levava a fita de saída para um computador 1401 imprimir-la off-line, isto é, não conectada ao computador principal.

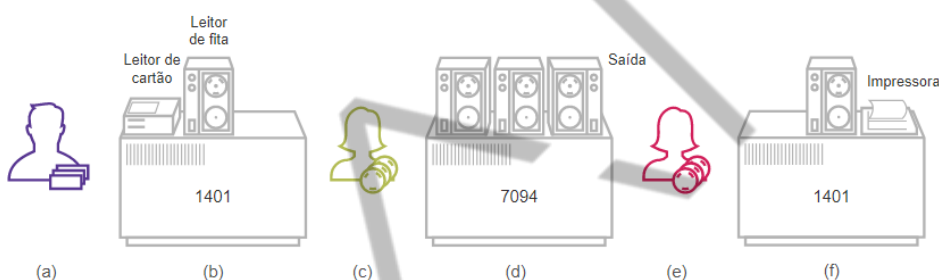


Figura 7 – Sistema em lote (batch) antigo na segunda geração
Fonte: Tanenbaum (2009), adaptado por Fiap (2020)

Segundo a figura “Sistema em lote (batch) antigo na segunda geração”, em (a) Os programadores levavam os cartões para o 1401. (b) O 1401 gravava o lote de tarefas em fita, (c) O operador levava a fita de entrada para o 70910. (d) O 7094 executava o processamento, (e) O operador levava a fita de saída para o 1401. (f) O 1401 imprimia as saídas.

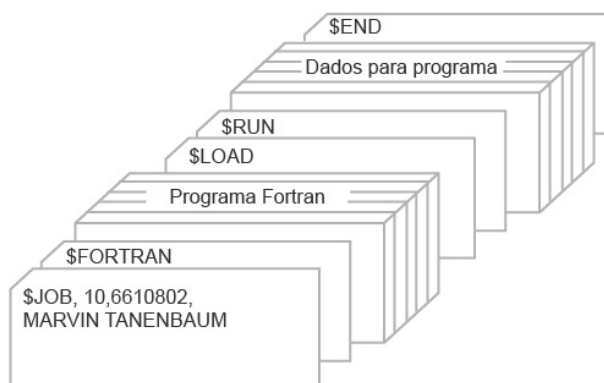


Figura 8 – Estrutura de uma tarefa FMS
Fonte: Tanenbaum (2009), adaptado por Fiap (2020)

A estrutura de uma tarefa típica é mostrada na Figura “Estrutura de uma tarefa FMS”: ela começava com um cartão \$JOB, que especificava o tempo máximo de processamento em minutos, o número da conta a ser debitada e o número do programador. Em seguida, vinha um cartão SFORTRAN, que mandava o sistema operacional carregar o compilador Fortran a partir da fita de sistema. Depois, vinham os cartões do programa a ser compilado e, então, um cartão \$LOAD, que ordenava ao sistema operacional o carregamento do programa-objeto recém-compilado. (Os programas compilados, muitas vezes, eram gravados em fitas-rascunho e tinham de ser carregados explicitamente.)

Era então a vez do cartão \$RUN, que dizia para o sistema operacional executar o programa com o conjunto de dados constante nos cartões seguintes. Por fim, o cartão \$END marcava a conclusão da tarefa. Esses cartões de controle foram os precursores das linguagens de controle de tarefas e dos interpretadores de comando atuais.

Os grandes computadores de segunda geração foram usados, em sua maioria, para cálculos científicos, como equações diferenciais parciais, muito frequentes na física e na engenharia. Eles eram preponderantemente programados em Fortran e em linguagem Assembly. Os sistemas operacionais típicos eram o FMS (*Fortran Monitor System*) e o IBSYS, o sistema operacional da IBM para o 70910.

4.4 A terceira geração (1965-1980) - CIs e multiprogramação

No início na década de 1960, a maioria dos fabricantes de computador oferecia duas linhas de produtos distintas e totalmente incompatíveis. De um lado, havia os computadores científicos de grande escala e orientados a palavras, como o 7094, usados para cálculos numéricos na ciência e na engenharia. De outro, existiam os computadores comerciais orientados a caracteres, como o 1401, amplamente usados por bancos e companhias de seguros para ordenação e impressão em fitas.

Desenvolver e manter duas linhas de produtos completamente diferentes demandava grande custo para os fabricantes. Além disso, muitos dos clientes precisavam, inicialmente, de uma pequena máquina, mas depois expandiam seus negócios e, com isso, passavam a demandar máquinas maiores que pudessem executar todos os seus programas antigos, porém, mais rapidamente.

A IBM tentou resolver esses problemas de uma única vez, introduzindo o System/360. O 360 era uma série de máquinas — desde máquinas do porte do 1401 até as mais potentes que o 7094, cujos softwares eram compatíveis. Os equipamentos divergiam apenas no preço e no desempenho (quantidade máxima de memória, velocidade do processador, número de dispositivos de E/S permitidos etc.).

Como todas as máquinas tinham a mesma arquitetura e o mesmo conjunto de instruções, os programas escritos para uma máquina podiam ser executados em todas as outras, pelo menos teoricamente. Além disso, o 360 era voltado tanto para a computação científica (isto é, numérica) quanto para a comercial. Assim, uma única família de máquinas poderia satisfazer as necessidades de todos os clientes.

Nos anos subsequentes, a IBM lançou sucessivas séries compatíveis com a linha 360, usando tecnologias mais modernas, conhecidas como séries 370, 4300, 3080 e 3090. O *Z series* é o mais novo descendente dessa linha, embora tenha se afastado consideravelmente do original.

O IBM 360 foi a primeira linha de computadores a usar circuitos integrados (CIs) em pequena escala, propiciando, assim, uma melhor relação custo-benefício em comparação à segunda geração de máquinas, construídas com transistores individuais. Foi um sucesso instantâneo, e a ideia de uma família de computadores compatíveis logo foi adotada por todos os outros fabricantes.

Os descendentes dessas máquinas ainda estão em uso nos centros de computação. Atualmente, são mais empregados para gerenciar imensos bancos de dados (por exemplo, para sistemas de reservas aéreas) ou como servidores para sites da Web, que precisam processar milhares de requisições por segundo.

O forte da ideia de 'família de máquinas' era, simultaneamente, sua maior fraqueza. A intenção era que qualquer software, inclusive o sistema operacional OS/360, pudesse ser executado em qualquer um dos modelos. O software precisava ser executado em sistemas pequenos que geralmente apenas substituíam os 1401 na transferência de cartões perfurados para fita magnética e em sistemas muito grandes, que frequentemente substituíam os 7094 na previsão do tempo e em outras operações pesadas.

Tinha de ser eficiente tanto em sistemas com poucos periféricos como nos com muitos periféricos. Tinha de funcionar bem em ambientes comerciais e em ambientes científicos. E, acima de tudo, o sistema operacional precisava provar ser eficaz com todos esses diferentes usos.

Não havia como a IBM (ou qualquer outro fabricante) elaborar um software que resolvesse todos esses requisitos conflitantes. O resultado foi um sistema operacional enorme e extraordinariamente complexo, provavelmente duas ou três vezes maior que o FMS.

Eram milhões de linhas escritas em linguagem *Assembly* por milhares de programadores, contendo milhares de erros, que precisavam de um fluxo contínuo de novas versões para tentar corrigi-los. Cada nova versão corrigia alguns erros, mas introduzia outros, fazendo com que, provavelmente, o número de erros permanecesse constante ao longo do tempo.

Apesar de seu enorme tamanho e de seus problemas, o sistema OS/360 da IBM e os sistemas operacionais similares de terceira geração elaborados por outros fabricantes de computadores atendiam razoavelmente bem à maioria dos clientes. Também popularizavam várias técnicas fundamentais ausentes nos sistemas operacionais de segunda geração. A mais importante dessas técnicas foi, provavelmente, a multiprogramação.

No 7094, quando a tarefa atual parava para esperar por uma fita magnética terminar a transferência ou aguardava o término de outra operação de E/S, a CPU simplesmente permanecia ociosa até que a E/S terminasse. Para cálculos científicos com uso intenso do processador (*CPU-bound*), a E/S era pouco frequente, de modo que o tempo gasto com ela não era significativo. Para o processamento de dados comerciais, o tempo de espera pela E/S chegava a 80 ou 90 por cento do tempo total (*IO-bound*), de modo que algo precisava ser feito para evitar que a CPU ficasse ociosa todo esse tempo.

A solução a que se chegou foi dividir a memória em várias partes, com uma tarefa diferente em cada partição: enquanto uma tarefa esperava que uma operação de E/S se completasse, outra poderia usar a CPU. Se um número suficiente de tarefas pudesse ser mantido na memória ao mesmo tempo, a CPU poderia permanecer ocupada por quase 100 por cento do tempo.

Manter múltiplas tarefas de maneira segura na memória, por sua vez, requeria um hardware especial para proteger cada tarefa contra danos e transgressões causados por outras tarefas, mas o 360 e outros sistemas de terceira geração eram equipados com esse hardware.

Outro aspecto importante nos sistemas operacionais de terceira geração era a capacidade de transferir tarefas de cartões perfurados para discos magnéticos logo que esses chegassem à sala do computador. Dessa forma, assim que uma tarefa fosse completada, o sistema operacional poderia carregar uma nova tarefa a partir do disco na partição que acabou de ser liberada e, então, processá-la. Essa técnica é denominada *spooling* (termo derivado da expressão *simultaneous peripheral operation online*) e também foi usada para arbitrar a saída. Com o *spooling*, os 1401 não eram mais necessários, e muito do leva e traz de fitas magnéticas desapareceu.

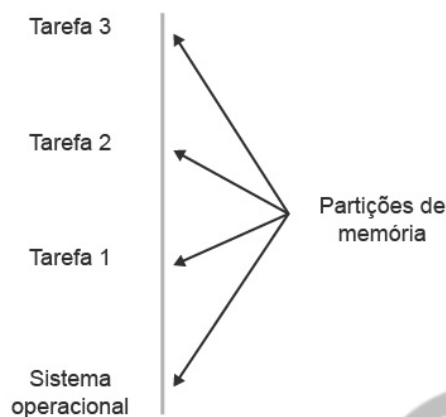


Figura 9 – Partições da Memória
Fonte: Tanenbaum (2009), adaptado por Fiap (2020)

Embora os sistemas operacionais de terceira geração fossem adequados para grandes cálculos científicos e processamento maciço de dados comerciais, eram basicamente sistemas em lote (*batch systems*). Muitos programadores sentiam saudades da primeira geração, quando podiam dispor da máquina por algumas horas, podendo, assim, corrigir seus programas mais rapidamente.

Com os sistemas de terceira geração, o intervalo de tempo entre submeter uma tarefa e obter uma saída normalmente era de muitas horas; assim, uma única vírgula errada poderia causar um erro de compilação, e o programador gastaria metade do dia para corrigi-lo.

O anseio por respostas mais rápidas abriu caminho para o tempo compartilhado ou *time sharing*, uma variante da multiprogramação, na qual cada usuário se conectava por meio de um terminal on-line. Em um sistema de tempo compartilhado, se 20 usuários estivessem conectados e 17 deles estivessem pensando, falando ou tomando café, a CPU podia ser ciclicamente alocada a cada uma das três tarefas que estivessem requisitando a CPU.

Como, ao depurar programas, emitem-se, normalmente, comandos curtos (por exemplo, compile uma rotina de cinco páginas) em vez de comandos longos (por exemplo, ordene um arquivo de um milhão de registros), o computador era capaz de fornecer um serviço rápido e interativo a vários usuários e ainda processar grandes lotes de tarefas em background (segundo plano) nos instantes em que a CPU estivesse ociosa.

O primeiro sistema importante de tempo compartilhado, o CTSS (*compatible time sharing system*), sistema compatível de tempo compartilhado, foi desenvolvido no MIT em um 7094 modificado. Contudo, o tempo compartilhado só se popularizou durante a terceira geração, período em que a necessária proteção em hardware foi largamente empregada.

Depois do sucesso desse sistema (CTSS), o MIT, o Bell Labs e a General Electric (então, um dos grandes fabricantes de computadores) decidiram desenvolver um 'computador utilitário', uma máquina que suportasse simultaneamente centenas de usuários compartilhando o tempo.

Basearam-se no modelo do sistema de distribuição de eletricidade, ou seja, quando se precisa de energia elétrica, conecta-se o pino na tomada da parede e, não havendo nenhum problema, tem-se tanta energia quanto é necessário. Os projetistas desse sistema, conhecido como MULTICS (*Multiplexed Information and computing Service* — serviço de computação e de informação multiplexada), imaginaram uma enorme máquina fornecendo 'energia' computacional para toda a área de Boston.

A ideia de que máquinas muito mais potentes que o computador de grande porte GE-645 fossem vendidas por mil dólares para milhões de pessoas, apenas 40 anos mais tarde, era ainda pura ficção científica. Seria como imaginar, hoje, trens submarinos transatlânticos e supersônicos.

O MULTICS foi projetado para suportar centenas de usuários em uma única máquina somente um pouco mais potente que um PC baseado no 386 da Intel, embora tendo muito mais capacidade de E/S. Isso não é tão absurdo quanto parece, já que, naqueles dias, sabia-se escrever programas pequenos e eficientes — uma habilidade que se vem perdendo a cada dia.

Havia muitas razões para que o MULTICS não dominasse o mundo; uma delas era sua codificação em PL/I. O compilador PL/I chegou com anos de atraso e, quando isso aconteceu, dificilmente funcionava. Além disso, o MULTICS era muito ambicioso para seu tempo, tanto quanto a máquina analítica de Charles Babbage no século XIX. Apesar disso tudo, podemos dizer que a ideia lançada pelo MULTICS foi bem-sucedida.

Para resumir a história, o MULTICS introduziu muitas ideias seminais na literatura da computação, mas torná-lo um produto sério e um grande sucesso comercial era muito mais difícil do que se pensava. O Bell Labs retirou-se do projeto e a General Electric saiu do negócio de computadores. Contudo, o MIT persistiu e finalmente fez o MULTICS funcionar. Ele foi, então, vendido como produto comercial pela empresa que comprou o negócio de computadores da GE (a Honeywell) e instalado em cerca de 80 grandes empresas e universidades pelo mundo.

Apesar de pouco numerosos, os usuários do MULTICS eram extremamente leais. A General Motors, a Ford e a Agência de Segurança Nacional dos Estados Unidos (U.S. National Security Agency), por exemplo, somente desligaram seus sistemas MULTICS no final dos anos 1990, três décadas depois de seu lançamento, após anos de tentativas para que a Honeywell atualizasse o *hardware*.

Por ora, o conceito de 'computador utilitário' permanece adormecido, mas pode muito bem ser trazido de volta na forma de poderosos servidores de Internet centralizados, nos quais máquinas usuárias mais simples são conectadas, com a maior parte do trabalho acontecendo nesses grandes servidores.

A motivação para isso é que, possivelmente, a maioria das pessoas não pretende administrar um sistema cuja complexidade é crescente e cuja operação torna-se cada vez mais meticulosa, assim, será preferível que essa tarefa seja realizada por uma equipe de profissionais trabalhando para a empresa que opera o servidor.

O comércio eletrônico (*e-commerce*) já está evoluindo nessa direção, com várias empresas no papel de centros comerciais eletrônicos (*e-malls*) em servidores multiprocessadores aos quais as máquinas dos clientes se conectam, no melhor espírito do projeto MULTICS.

Outro grande desenvolvimento ocorrido durante a terceira geração foi o fenomenal crescimento dos minicomputadores, iniciado com o DEC PDP-1 em 1961. O PDP-1 tinha somente 4 K de palavras de 18 bits, mas cada máquina custava 120 mil dólares (menos de 5 por cento do preço de um 7094) e, mesmo assim, vendia como água. Para certos tipos de aplicações não numéricas, era tão rápido quanto o 7094 e deu origem a toda uma nova indústria. Rapidamente foi seguido por uma série de outros PDPs (diferentemente da família IBM, todos incompatíveis), culminando no PDP-11.

Ken Thompson, um dos cientistas da computação do Bell Labs que trabalharam no projeto MULTICS, achou um pequeno minicomputador PDP-7 que ninguém estava usando e aproveitou-o para escrever uma versão despojada e monousuário do MULTICS. Esse trabalho desenvolveu-se e deu origem ao sistema operacional UNIX®, que se tornou muito popular no mundo acadêmico, em agências governamentais e em muitas empresas.

O UNIX® se tornou popular, muito em virtude de o código-fonte ter sido amplamente divulgado, várias organizações desenvolveram suas próprias (e incompatíveis) versões, o que levou ao caos. Duas das principais versões desenvolvidas, o System V, da AT&T, e o BSD (*Berkeley software distribution* — distribuição de software de Berkeley), da Universidade da Califórnia em Berkeley, também possuíam variações menores.

Para tornar possível escrever programas que pudessem ser executados em qualquer sistema UNIX, o IEEE desenvolveu um padrão para o UNIX denominado POSIX (*portable operating System interface* — interface portátil para sistemas operacionais), ao qual a maioria das versões UNIX agora dá suporte.

O POSIX define uma interface mínima de chamada de sistema a que os sistemas em conformidade com o UNIX devem dar suporte. Na verdade, alguns outros sistemas operacionais agora também dão suporte à interface POSIX.

4.5 A quarta geração (1980-presente) - computadores pessoais

Com o desenvolvimento de circuitos integrados em larga escala (*large scale integration* — LSI), que são chips contendo milhares de transistores em um centímetro quadrado de silício, surgiu a era dos computadores pessoais. Em termos de arquitetura, os computadores pessoais (inicialmente denominados microcomputadores) não eram muito diferentes dos minicomputadores da classe PDP-11, mas no preço eram claramente diferentes. Se o minicomputador tornou possível para um departamento, uma empresa ou uma universidade ter seu próprio computador, o chip microprocessador tornou possível a um indivíduo qualquer ter seu próprio computador pessoal.

Em 1974, a Intel lançou o 8080, a primeira CPU de 8 bits de uso geral, e buscava um sistema operacional para o 8080, em parte para testá-lo. A Intel pediu a um de seus consultores, Gary Kildall, para escrevê-lo. Kildall e um amigo inicialmente construíram um controlador para a então recém-lançada unidade de discos flexíveis de 8 polegadas da Shugart Associates e a utilizaram com um 8080, produzindo, assim, o primeiro microcomputador com unidade de discos flexíveis.

Kildall então escreveu para ele um sistema operacional baseado em disco, denominado CP/M (*Control Program for Microcomputers* — programa de controle para microcomputadores). Como a Intel não acreditava que microcomputadores baseados em disco tivessem muito futuro, Kildall requisitou os direitos sobre o CP/M e a Intel os cedeu. Kildall formou, então, uma empresa, a Digital Research, para aperfeiçoar e vender o CP/M.

Em 1977, a *Digital Research* reescreveu o CP/M para torná-lo adequado à execução em muitos microcomputadores utilizando 8080, Zilog Z80 e outros microprocessadores. Muitos programas aplicativos foram escritos para serem executados no CP/M, permitindo que ele dominasse completamente o mundo da microcomputação por cerca de cinco anos.

No início dos anos 1980, a IBM projetou o IBM PC e buscou um software para ser executado nele. O pessoal da IBM entrou em contato com Bill Gates para licenciar seu interpretador Basic. Também lhe foi indagado se ele conhecia algum sistema operacional que pudesse ser executado no PC. Gates sugeriu que a IBM contatasse a Digital Research, a empresa que dominava o mundo dos sistemas operacionais naquela época.

Tomando seguramente a pior decisão de negócios registrada na história, Kildall recusou-se a se reunir com a IBM, enviando em seu lugar um subordinado. Para piorar, o advogado dele foi contra assinar um acordo de sigilo sobre o PC que ainda não havia sido divulgado. Consequentemente, a IBM voltou a Gates, perguntando-lhe se seria possível fornecer-lhes um sistema operacional.

Então Gates percebeu que uma fabricante local de computadores, a *Seattle Computer Products*, possuía um sistema operacional adequado, o DOS (*disk operating system* — sistema operacional de disco). Entrou em contato com essa empresa e disse que queria comprá-la (supostamente por 75 mil dólares), o que foi prontamente aceito.

Gates ofereceu à IBM um pacote DOS/Basic, e ela aceitou. A IBM quis fazer algumas modificações, e para isso Gates contratou a pessoa que tinha escrito o DOS, Tim Paterson, como funcionário da empresa embrionária de Gates, a Microsoft.

O sistema revisado teve seu nome mudado para MS-DOS (*Microsoft disk operating system* — sistema operacional de disco da Microsoft) e rapidamente viria a dominar o mercado do IBM PC. Um fator decisivo para isso foi a decisão de Gates (agora, olhando o passado, extremamente sábia) de vender o MS-DOS para empresas de computadores acompanhando o hardware, em vez de tentar vender diretamente aos usuários finais (pelo menos inicialmente), como Kildall tentou fazer com o CP/M. Depois de todos esses acontecimentos, Kildall morreu repentina e inesperadamente de causas que não foram completamente reveladas.

Quando, em 1983, o sucessor do IBM PC, o IBM PC/AT, foi lançado utilizando a CPU Intel 80286, o MS-DOS avançava firmemente, ao mesmo tempo em que o CP/M definhava. O MS-DOS foi, mais tarde, também amplamente usado com o 80386 e o 80486. Mesmo com uma versão inicial bastante primitiva, as versões subsequentes do MS-DOS incluíram aspectos mais avançados, muitos deles derivados do UNIX. (A Microsoft conhecia bem o UNIX, pois, nos primeiros anos da empresa, vendeu uma versão para microcomputadores do UNIX, denominada XENIX).

O CP/M, o MS-DOS e outros sistemas operacionais dos primeiros microcomputadores eram todos baseados na digitação de comandos em um teclado, feita pelo usuário. Isso finalmente mudou graças a um trabalho de pesquisa de Doug Engelbart no *Stanford Research Institute* nos anos 1960. Engelbart inventou uma interface gráfica completa — voltada para o usuário — com janelas, ícones, menus e mouse, denominada GUI (*graphical user interface*). Essas ideias de Engelbart foram adotadas por pesquisadores da PARC (*Palo Alto Research Center*), subsidiária da Xerox, e foram incorporadas às máquinas que eles projetaram.

Um dia, Steve Jobs, que co-inventou o computador Apple na garagem de sua casa, visitou a XEROX PARC, viu uma interface gráfica GUI e instantaneamente percebeu seu potencial, algo que a gerência da Xerox reconhecidamente não tinha feito. Jobs, então, iniciou a construção de um Apple dispondo de uma interface gráfica GUI. Esse projeto, denominado Lisa, foi muito dispendioso e falhou comercialmente.

A segunda tentativa de Jobs, o Apple Macintosh, foi um enorme sucesso, não somente por seu custo muito menor que o do Lisa, mas também porque era mais amigável ao usuário, destinado a usuários que não só nada sabiam sobre computadores, mas também não tinham a menor intenção de um dia aprender sobre eles. No mundo criativo do design gráfico, da fotografia digital profissional e da produção profissional de vídeos digitais, os computadores Macintosh são amplamente empregados e os usuários são seus grandes entusiastas.



Figura 10 – Jovem Steve Jobs ilustrando uma parede do museu da Apple em Praga
Fonte: Shutterstock (2017)

Quando a Microsoft decidiu elaborar um sucessor para o MS-DOS, estava fortemente influenciada pelo sucesso do Macintosh. Desenvolveu um sistema denominado Windows, baseado na interface gráfica GUI, que era executado originalmente em cima do MS-DOS (isto é, era como se fosse um interpretador de comandos Shell em vez de um sistema operacional de verdade).

Por aproximadamente dez anos, de 1985 a 1995, o Windows permaneceu apenas como um ambiente gráfico sobre o MS-DOS. Contudo, em 1995, lançou-se uma versão do Windows independente do MS-DOS, o Windows 95. Nessa versão, o Windows incorporou muitos aspectos de um sistema operacional, usando o MS-DOS apenas para ser carregado e executar programas antigos do MS-DOS.

Em 1998, lançou-se uma versão levemente modificada desse sistema, chamada Windows 98. No entanto, ambos, o Windows 95 e o Windows 98, ainda continham uma grande quantidade de código em linguagem Assembly de 16 bits da Intel.



Figura 11 – Discos flexíveis 3 ½ polegadas necessários para instalar o Windows 95
Fonte: Shutterstock (2019)

Outro sistema operacional da Microsoft é o Windows NT (NT é uma sigla para *new technology*), que é compatível com o Windows 95 em certo nível, mas reescrito internamente por completo. É um sistema de 32 bits completo. O líder do projeto do Windows NT foi David Cutler, que foi também um dos projetistas do sistema operacional VAX VMS, e, por isso, algumas ideias do VMS estão presentes no NT.

Na realidade, havia tantas ideias do VMS no sistema que a proprietária do VMS, a DEC, processou a Microsoft. As partes entraram em acordo sobre o caso por uma enorme quantia de dinheiro. A Microsoft esperava que a primeira versão do NT 'aposentasse' o MS-DOS e todas as outras versões do Windows, já que o NT era muito superior, mas isso não aconteceu.

Somente com a versão Windows NT 10.0 foi que ele finalmente deslanchou, especialmente em redes corporativas. A versão 5 do Windows NT foi renomeada Windows 2000 no início de 1999. Seu objetivo era suceder tanto o Windows 98 quanto o Windows NT 10.0.

Essa versão também não obteve êxito, então a Microsoft lançou mais uma versão do Windows 98, denominada Windows Me (Millennium Edition). Em 2001, uma versão ligeiramente atualizada do Windows 2000, chamada Windows XP, foi lançada. Essa versão teve duração muito maior (seis anos), substituindo basicamente todas as versões anteriores do Windows.

Em janeiro de 2007, a Microsoft finalmente lançou o sucessor do Windows XP, chamado Vista. Ele apresentou uma nova interface gráfica, Aero, e muitos programas de usuário novos ou atualizados.

Parte da história envolvendo os bastidores da criação da Apple e Microsoft pode ser conferida no telefilme “Piratas do Vale do Silício” (1999).

5 TIPOS DE SISTEMAS OPERACIONAIS

Os sistemas operacionais existem há mais de 50 anos. Durante esse tempo, uma grande variedade deles foi desenvolvida, nem todos bem conhecidos. Nesta seção, falaremos resumidamente sobre nove deles. Mais adiante, voltaremos a abordar alguns desses diferentes tipos de sistemas.

5.1 Sistemas operacionais de computadores de grande porte

No topo estão os sistemas operacionais para computadores de grande porte, aqueles que ocupam uma sala inteira, ainda encontrados em centros de dados de grandes corporações. Esses computadores distinguem-se dos computadores pessoais em termos de capacidade de E/S.

Um computador de grande porte, com mil discos e milhares de gigabytes de dados, não é incomum; um computador pessoal com essas especificações seria algo singular.

Os computadores de grande porte também estão ressurgindo como sofisticados servidores da Web, como servidores para sites de comércio eletrônico em larga escala e, ainda, como servidores para transações entre empresas (*business-to-business*).

Os sistemas operacionais para computadores de grande porte são, sobretudo, orientados para o processamento simultâneo de muitas tarefas, e a maioria deles precisa de quantidades prodigiosas de E/S. Esses sistemas operacionais normalmente oferecem três tipos de serviços: em lote (*batch*), processamento de transações e tempo compartilhado.

Um sistema em lote processa tarefas de rotina sem a presença interativa do usuário. O processamento de apólices de uma companhia de seguros ou de relatórios de vendas de uma cadeia de lojas é, em geral, realizado em modo lote. Sistemas de processamento de transações administram grandes quantidades de pequenas requisições, por exemplo, processamento de verificações em um banco ou em reservas de passagens aéreas. Cada unidade de trabalho é pequena, mas o sistema precisa tratar centenas ou milhares delas por segundo.

Sistemas de tempo compartilhado permitem que múltiplos usuários remotos executem suas tarefas simultaneamente no computador, como na realização de consultas a um grande banco de dados. Essas funções estão intimamente relacionadas; sistemas operacionais de computadores de grande porte muitas vezes realizam todas elas. Um exemplo de sistema operacional de computador de grande porte é o OS/390, um descendente do OS/360. Entretanto, os sistemas operacionais de computadores de grande porte estão sendo gradualmente substituídos por variantes do UNIX, como o Linux.

5.2 Sistemas operacionais de servidores

Um nível abaixo estão os sistemas operacionais de servidores. Eles são executados em servidores, que são computadores pessoais muito grandes, em estações de trabalho ou até mesmo em computadores de grande porte.

Eles servem múltiplos usuários de uma vez, em uma rede, e permitem-lhes compartilhar recursos de hardware e de software. Servidores podem fornecer serviços de impressão, de arquivo ou de Web.

Provedores de acesso à Internet utilizam várias máquinas servidoras para dar suporte a seus clientes e sites da Web usam servidores para armazenar páginas e tratar requisições que chegam. Sistemas operacionais típicos de servidores são Solaris, FreeBSD, Linux e Windows Server 200x.

5.3 Sistemas operacionais de multiprocessadores

Um modo cada vez mais comum de obter potência computacional é conectar múltiplas CPUs em um único sistema. Dependendo precisamente de como elas estiverem conectadas e o que é compartilhado, esses sistemas são denominados computadores paralelos, multicomputadores ou multiprocessadores.

Elas precisam de sistemas operacionais especiais, mas muitos deles são variações dos sistemas operacionais de servidores, com aspectos especiais de comunicação, conectividade e compatibilidade.

Com o advento recente de chips multinúcleo para computadores pessoais, até sistemas operacionais de computadores de mesa e de notebooks estão começando a lidar com, no mínimo, multiprocessadores de pequena escala e é provável que o número de núcleos cresça com o tempo.

Felizmente, sabe-se bastante sobre sistemas operacionais de multiprocessadores como consequência de anos de pesquisas anteriores; desse modo, aplicar esse conhecimento a sistemas multinúcleo não deve ser difícil. A parte difícil seria fazer com que as aplicações usassem todo esse poder de computação. Muitos sistemas operacionais populares, inclusive Windows e Linux, são executados com multiprocessadores.

5.4 Sistemas operacionais de computadores pessoais

A categoria seguinte é o sistema operacional de computadores pessoais. Os computadores modernos dão suporte à multiprogramação, muitas vezes com dezenas de programas iniciados. Seu trabalho é oferecer uma boa interface para um único usuário.

São amplamente usados para processadores de texto, planilhas e acesso à Internet. Exemplos comuns são Linux, Microsoft Windows e o Mac OS X. Sistemas operacionais de computadores pessoais são tão amplamente conhecidos que é provável que precisem, aqui, de pouca introdução. Na verdade, muitas pessoas nem mesmo sabem da existência de outros tipos de sistemas operacionais.

5.5 Sistemas operacionais de computadores portáteis

Seguindo em direção a sistemas cada vez menores, chegamos aos computadores portáteis. Um computador portátil ou assistente pessoal digital (*personal digital assistant* — PDA) era um pequeno computador que cabia no bolso de uma camisa e executava um número pequeno de funções, como agenda de endereços e bloco de anotações.

Nos dias de hoje, os chamados PDAs e telefones celulares basicamente se fundiram, diferindo principalmente em tamanho, peso e interface com o usuário, tornando-se *smartphones*. Quase todos eles são baseados em CPUs de 32 bits com modo protegido e executam um sistema operacional sofisticado.

Os sistemas operacionais executados nesses computadores portáteis são cada vez mais sofisticados, com a capacidade de manipular telefonia, fotografia digital e outras funções. Tínhamos, no passado, os sistemas Symbian OS (difundido principalmente pela fabricante de celular Nokia) e Palm OS (dos antigos Palm Tops). Hoje, o mercado se divide basicamente em Android (baseado em Linux) e o iOS (da Apple).



Figura 12 – Um antigo e saudoso Palm Top, caso você nunca tenha visto um
Fonte: Shutterstock (2019)

5.6 Sistemas operacionais embarcados

Sistemas embarcados são executados em computadores que controlam dispositivos que geralmente não são considerados computadores e que não aceitam softwares instalados por usuários. Exemplos típicos são fornos de micro-ondas, smartTVs, carros, aparelhos de DVD, telefones celulares e reprodutores de MP3.

A propriedade principal que distingue os sistemas embarcados dos portáteis é a certeza de que nenhum software não confiável jamais será executado nele. Você não pode baixar novas aplicações para seu forno de micro-ondas, todo software está no ROM. Isso significa que não há necessidade de proteção entre as aplicações, levando a algumas simplificações. Sistemas como QNX e VxWorks são populares nesse domínio.



Figura 13 – Máquina de lavar roupa também tem sistema embarcado, sabia?
Fonte: Shutterstock (2019)

5.7 Sistemas operacionais de nós sensores (*sensor node*)

Redes de nós sensores minúsculos estão sendo empregadas com inúmeras finalidades. Esses nós são computadores minúsculos que se comunicam entre si e com uma estação-base usando comunicação sem fio. Essas redes de sensores são utilizadas para proteger os perímetros de prédios, guardar fronteiras nacionais, detectar incêndios em florestas, medir temperatura e níveis de precipitação para previsão do tempo, colher informações sobre movimentos dos inimigos em campos de batalha e muito mais.

Os sensores são computadores pequenos, movidos a bateria, com rádios integrados. Eles têm energia limitada e devem funcionar por longos períodos de tempo, sozinhos ao ar livre, frequentemente em condições ambientais severas. A rede deve ser robusta o suficiente para tolerar falhas de nós individuais, o que acontece com frequência cada vez maior, à medida que as baterias começam a se esgotar.

Cada nó sensor é um verdadeiro computador, com CPU, RAM, ROM e um ou mais sensores ambientais. Executa um pequeno sistema operacional próprio, normalmente dirigido por eventos, reagindo a eventos externos ou obtendo medidas periodicamente, com base em um relógio interno.

O sistema operacional tem de ser pequeno e simples, porque os nós têm RAM pequena e a duração da bateria é uma questão importante. Além disso, tal como nos sistemas embarcados, todos os programas são carregados antecipadamente; os usuários não iniciam repentinamente os programas que baixaram da Internet, o que torna o projeto muito mais simples.

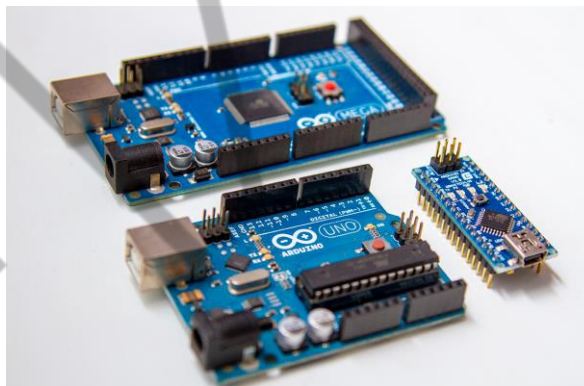


Figura 14 – Placas de prototipação Arduino, ideais para sensores modernos
Fonte: Shutterstock (2019)

5.8 Sistemas operacionais de tempo real

Outro tipo de sistema operacional é o de tempo real. Esses sistemas são caracterizados por terem o tempo como um parâmetro fundamental. Por exemplo, em sistemas de controle de processos industriais, computadores de tempo real devem coletar dados sobre o processo de produção e usá-los para controlar as máquinas na fábrica.

É bastante comum a existência de prazos rígidos para a execução de determinadas tarefas. Por exemplo, se um carro está se movendo por uma linha de montagem, certas ações devem ser realizadas em momentos específicos.

Se um robô soldador, ao realizar seu trabalho, soldar muito cedo ou muito tarde, o carro estará perdido. Se as ações precisam necessariamente ocorrer em determinados instantes (ou em um determinado intervalo de tempo), tem-se, então, um sistema de tempo real crítico.

Muitos deles são encontrados no controle de processos industriais, aviação, exército e áreas de aplicação semelhantes. Esses sistemas devem fornecer garantia absoluta de que determinada ação ocorrerá em determinado momento.



Figura 15 – Já imaginou um sistema de UTI dando tela azul da morte? Não pode, né?
Fonte: Shutterstock (2019)

Outro tipo de sistema de tempo real é o sistema de tempo real não crítico, no qual o descumprimento ocasional de um prazo, embora não desejável, é aceitável e não causa nenhum dano permanente. Sistemas de áudio digital ou multimídia pertencem a essa categoria.

5.9 Sistemas operacionais de cartões inteligentes (*smart cards*)

Os menores sistemas operacionais são executados em cartões inteligentes, dispositivos do tamanho de cartões de crédito que contêm um chip de CPU. Possuem grandes restrições de consumo de energia e de memória. Alguns deles obtêm energia por contatos com o leitor em que estão inseridos, porém, os cartões inteligentes sem contato obtêm energia por indução, o que limita muito aquilo que podem realizar.

Alguns deles podem realizar apenas uma única função, como pagamentos eletrônicos, mas outros podem gerenciar múltiplas funções no mesmo cartão inteligente. São comumente sistemas proprietários.

Alguns cartões inteligentes são orientados a Java. Isso significa que a ROM no cartão inteligente contém um interpretador para a máquina virtual Java (*Java virtual machine* — JVM). As pequenas aplicações Java (*applets*) são carregadas no cartão e interpretadas pela JVM.

Alguns desses cartões podem tratar múltiplas *applets* Java ao mesmo tempo, acarretando multiprogramação e a consequente necessidade de escalonamento. O gerenciamento de recursos e a proteção também são um problema, quando duas ou mais *applets* estão presentes simultaneamente. Esses problemas devem ser tratados pelo sistema operacional (normalmente muito primitivo) contido no cartão.

Falemos agora de conceitos importantes referentes aos sistemas operacionais.

6 THREADS

Em sistemas operacionais tradicionais, cada processo tem um espaço de endereçamento e um único thread de controle. Na verdade, isso é quase uma definição de processo. Contudo, frequentemente há situações em que é desejável ter múltiplos threads de controle no mesmo espaço de endereçamento, executando em quase-paralelo, como se eles fossem processos separados (exceto pelo espaço de endereçamento compartilhado). A seguir, discutiremos essas situações e suas implicações.

6.1 O uso da Thread

Por que alguém desejaria ter um tipo de processo dentro de um processo? Consta-se que há várias razões para existirem esses miniprocessos, chamados *threads*. Examinemos alguns deles agora. A principal razão para existirem *threads* é que em muitas aplicações ocorrem múltiplas atividades ao mesmo tempo.

Algumas dessas atividades podem ser bloqueadas de tempos em tempos. O modelo de programação se torna mais simples ao decompor uma aplicação em múltiplos threads sequenciais que executam quase em paralelo.

Já vimos esse argumento antes. É precisamente o mesmo argumento para a existência dos processos. Em vez de pensarmos em interrupções, temporizadores e chaveamento de contextos, podemos pensar em processos paralelos. Só que agora, com os *threads*, adicionamos um novo elemento: a capacidade de entidades paralelas compartilharem um espaço de endereçamento e todos os seus dados entre elas. Isso é essencial para certas aplicações, nas quais múltiplos processos (com seus espaços de endereçamento separados) não funcionarão.

Um segundo argumento para a existência de *threads* é que são mais fáceis (isto é, mais rápidos) de criar e destruir que os processos, pois não têm quaisquer recursos associados a eles. Em muitos sistemas, criar um *thread* é cem vezes mais rápido do que criar um processo. É útil ter essa propriedade quando o número de threads necessários se altera dinâmica e rapidamente.

Uma terceira razão é também um argumento de desempenho: o uso de *threads* não resulta em ganho de desempenho quando todos eles são *CPU-bound* (limitados pela CPU, isto é, muito processamento com pouca E/S). No entanto, quando há grande quantidade de computação e de E/S, os threads permitem que essas atividades se sobreponham e, desse modo, aceleram a aplicação.

A maneira mais fácil de perceber a utilidade dos *threads* é apresentar exemplos concretos. Como primeiro exemplo, considere um processador de textos. A maioria dos processadores de texto mostra o documento em criação na tela, formatado exatamente como ele aparecerá em uma página impressa.

Mais especificamente, todas as quebras de linha e de página estão na posição correta e final, para que o usuário possa conferi-las e alterar o documento, se for necessário (por exemplo, eliminar linhas viúvas e órfãs — linhas incompletas no início e no final de uma página, que são consideradas esteticamente desagradáveis).

Suponha que o usuário esteja escrevendo um livro. Do ponto de vista do autor, é mais fácil manter o livro inteiro como um arquivo único para tornar mais fácil a busca por tópicos, realizar substituições gerais e assim por diante. Mas há a alternativa de cada capítulo constituir um arquivo separado. Contudo, ter cada seção e subseção como um arquivo separado constitui um sério problema quando é necessário fazer alterações globais em todo o livro, já que, para isso, centenas de arquivos deverão ser editados individualmente.

Por exemplo, se um padrão proposto xxxx é aprovado um pouco antes de o livro seguir para impressão, todas as ocorrências de “Padrão Provisório xxxx” devem ser alteradas para “Padrão xxxx” no último minuto. Se o livro inteiro estiver em um arquivo, em geral, um único comando poderá fazer todas as substituições. Por outro lado, se o livro estiver dividido em 300 arquivos, cada um deles deverá ser editado separadamente.

Agora, imagine o que acontece quando o usuário remove, de repente, uma sentença da página 1 de um documento de 800 páginas. Depois de verificar a página alterada para se assegurar de que esteja correta, o usuário agora quer fazer outra mudança na página 600 e digita um comando dizendo para o processador de textos ir até aquela página (possivelmente buscando uma frase que apareça somente lá).

O processador de textos é, então, forçado a reformatar todo o conteúdo até a página 600, uma situação difícil, porque ele não sabe qual será a primeira linha da página 600 enquanto não tiver processado todas as páginas anteriores. Haverá uma demora substancial antes que a página 600 possa ser mostrada, deixando o usuário descontente.

Threads, nesse caso, podem ajudar. Suponha que o processador de textos seja escrito como um programa de dois *threads*. Um *thread* interage com o usuário e o outro faz a reformatação em segundo plano. Logo que uma sentença é removida da página 1, o *thread* interativo diz ao *thread* de reformatação para reformatar todo o livro. Enquanto isso, o *thread* interativo continua atendendo ao teclado, ao mouse e aos comandos simples, como rolar a página 1, enquanto o outro *thread* está processando a todo vapor em segundo plano. Com um pouco de sorte, a reformatação terminará antes que o usuário peça para ver a página 600, e, assim, ela poderá ser mostrada instantaneamente.

Enquanto estamos nesse exemplo, por que não adicionar um terceiro *thread*? Muitos processadores de texto são capacitados para salvar automaticamente todo o arquivo no disco a cada intervalo de tempo em minutos, a fim de proteger o usuário contra a perda de um dia de trabalho, caso ocorra uma falha no programa ou no sistema, ou mesmo uma queda de energia. O terceiro *thread* pode fazer os *backups* em disco sem interferir nos outros dois.

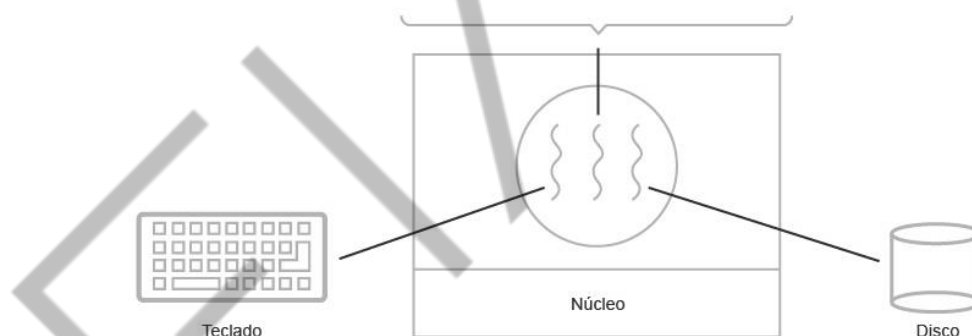


Figura 16 – Funcionamento das *threads*
Fonte: TANENBAUM (2009)

Se o programa tivesse apenas um *thread*, então, sempre que um *backup* de disco se iniciasse, os comandos do teclado e do mouse seriam ignorados enquanto o *backup* não terminasse. O usuário certamente perceberia isso como uma queda de desempenho.

Por outro lado, os eventos do teclado e do mouse poderiam interromper o *backup* em disco, permitindo um bom desempenho, mas levando a um complexo modelo de programação orientado à interrupção. Com três *threads*, o modelo de programação fica muito mais simples.

O primeiro *thread* apenas interage com o usuário; o segundo reformata o documento quando pedido; o terceiro escreve periodicamente o conteúdo da RAM no disco.

Deve estar claro que três processos separados não funcionariam no exemplo dado, pois todos os três threads precisam operar sobre o documento. Em vez de três processos, são três *threads* que compartilham uma memória comum e, desse modo, têm todo o acesso ao documento que está sendo editado.

Uma situação análoga ocorre com muitos outros programas interativos. Por exemplo, uma planilha eletrônica é um programa que permite que um usuário mantenha uma matriz, na qual alguns elementos são dados fornecidos pelo usuário. Outros elementos são calculados com base nos dados de entrada, usando-se fórmulas potencialmente complexas.

Quando um usuário altera um elemento, muitos outros elementos poderão vir a ser recalculados. Já que existe um *thread* em segundo plano para fazer o recálculo, o *thread* interativo pode possibilitar ao usuário fazer alterações adicionais enquanto a computação prossegue. Da mesma maneira, um terceiro *thread* pode cuidar dos *backups* periódicos para o disco.

7 PROCESSO

Todos os computadores modernos são capazes de fazer várias coisas ao mesmo tempo. As pessoas acostumadas a trabalhar com computadores pessoais podem não estar completamente cientes desse fato; portanto, alguns exemplos podem torná-lo mais claro.

Primeiro, considere um servidor da Web. Solicitações de páginas da *web* chegam de toda parte. Quando uma solicitação chega, o servidor verifica se a página necessária está na cache. Se estiver, é enviada de volta; se não, uma solicitação de acesso ao disco é iniciada para buscá-la.

Entretanto, do ponto de vista da CPU, as solicitações de acesso ao disco duram uma eternidade. Enquanto espera que a solicitação de acesso ao disco seja concluída, muitas outras solicitações podem chegar. Se há múltiplos discos presentes, algumas delas ou todas elas podem ser enviadas rapidamente a outros discos muito antes de a primeira solicitação ser atendida.

Evidentemente, é necessário algum modo de modelar e controlar essa simultaneidade. Os processos (e especialmente os threads) podem ajudar aqui.

Agora, considere um usuário de PC. Quando o sistema é inicializado, muitos processos, muitas vezes desconhecidos do usuário, começam secretamente. Por exemplo, um processo pode ser iniciado para espera de *e-mails* que chegam.

Outro processo pode ser executado pelo programa de antivírus para verificar periodicamente se há novas definições de antivírus disponíveis. Além disso, processos de usuários explícitos podem estar sendo executados, imprimindo arquivos e gravando um CD-ROM, tudo enquanto o usuário está navegando na Web. Toda essa atividade tem de ser administrada, e um sistema multiprogramado que sustente múltiplos processos é bastante útil nesse caso.

Em qualquer sistema multiprogramado, a CPU chaveia de programa para programa, executando cada um deles por dezenas ou centenas de milissegundos. Estritamente falando, enquanto a cada instante a CPU executa somente um programa, no decorrer de um segundo ela pode trabalhar sobre vários programas, dando aos usuários a ilusão de paralelismo.

Algumas vezes, nesse contexto, fala-se de pseudoparalelismo, para contrastar com o verdadeiro paralelismo de hardware dos sistemas multiprocessadores (que têm duas ou mais CPUs que compartilham simultaneamente a mesma memória física).

Ter controle sobre múltiplas atividades em paralelo é algo difícil para as pessoas. Contudo, projetistas de sistemas operacionais vêm desenvolvendo, ao longo dos anos, um modelo conceitual (processos sequenciais) que facilita o paralelismo. Esse modelo, seu uso e algumas de suas consequências compõem o assunto deste capítulo.

7.1 O modelo de processo

Neste modelo, todos os softwares que podem ser executados em um computador — inclusive, algumas vezes, os próprios sistemas operacionais — são organizados em vários processos sequenciais (ou, para simplificar, processos).

Um processo é apenas um programa em execução, acompanhado dos valores atuais do contador de programa, dos registradores e das variáveis. Conceitualmente, cada processo tem sua própria CPU virtual. É claro que, na realidade, a CPU troca, a todo o momento, de um processo para outro, mas, para entender o sistema, é muito mais fácil pensar em um conjunto de processos executando (*pseudo*) paralelamente do que tentar controlar o modo como a CPU faz esses chaveamentos. Esse mecanismo de trocas rápidas é chamado de multiprogramação.

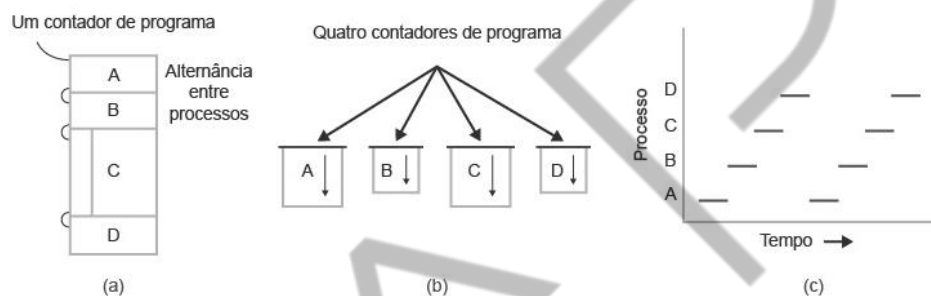


Figura 17 – Dinâmica dos processos
Fonte: TANENBAUM (2009)

Na Figura “Dinâmica dos processos”, em (a), vemos um computador multiprogramado com quatro programas na memória; em (b), estão quatro processos, cada um com seu próprio fluxo de controle (isto é, seu próprio contador de programa lógico) e executando independentemente dos outros.

Claro, há somente um contador de programa físico, de forma que, quando cada processo é executado, seu contador de programa lógico é carregado no contador de programa real. Quando acaba o tempo de CPU alocado para um processo, o contador de programa físico é salvo no contador de programa lógico do processo na memória.

Em (c), vemos que, por um intervalo de tempo suficientemente longo, todos os processos estão avançando, mas, a cada instante, um único processo está realmente executando.

Para efeito acadêmico, vamos supor que haja apenas uma CPU, embora na prática essa suposição não seja verdadeira, visto que os novos *chips* são muitas vezes multinúcleo (*multicore*), com duas, quatro ou mais CPUs.

Contudo, para explicação posterior, é mais simples pensar em uma CPU de cada vez. Assim, quando dizemos que uma CPU pode, de fato, executar apenas um processo por vez, se houver dois núcleos (ou duas CPUs), cada um deles pode executar apenas um processo por vez.

Com o rápido chaveamento da CPU entre os processos, a taxa na qual o processo realiza sua computação não será uniforme e provavelmente não será nem reproduzível, se os mesmos processos forem executados novamente.

Desse modo, os processos não devem ser programados com hipóteses predefinidas sobre a temporização. Considere, por exemplo, um processo de E/S que inicie uma fita magnética para que sejam restaurados arquivos de backup; ele executa dez mil vezes um laço ocioso para aguardar que uma rotação seja atingida e então executa um comando para ler o primeiro registro.

Se a CPU decidir chavear para outro processo durante a execução do laço ocioso, o processo da fita pode não estar sendo executado quando a cabeça de leitura chegar ao primeiro registro.

Quando um processo tem restrições críticas de tempo real como essas, isto é, eventos específicos devem ocorrer dentro de um intervalo de tempo prefixado de milissegundos, é preciso tomar medidas especiais para que esses eventos ocorram. Contudo, em geral, a maioria dos processos não é afetada pelo aspecto inerente de multiprogramação da CPU ou pelas velocidades relativas dos diversos processos.

A diferença entre um processo e um programa é sutil, mas crucial, e a analogia a seguir pode ajudar: imagine um cientista da computação com dotes culinários e que esteja assando um bolo de aniversário para sua filha. Ele tem uma receita de bolo de aniversário e uma cozinha bem suprida, com todos os ingredientes: farinha, ovos, açúcar, essência de baunilha, entre outros.

Nessa analogia, a receita é o programa (isto é, um algoritmo expresso por uma notação adequada), o cientista é o processador (CPU) e os ingredientes do bolo são os dados de entrada. O processo é a atividade desempenhada pelo nosso confeitiro de ler a receita, buscar os ingredientes e assar o bolo.

Agora, imagine que o filho do cientista chegue chorando, dizendo que uma abelha o picou. O cientista registra onde ele estava na receita (o estado atual do processo é salvo), busca um livro de primeiros socorros e começa a seguir as instruções contidas nele.

Nesse ponto, vemos que o processador está sendo alternado de um processo (assar o bolo) para um processo de prioridade mais alta (fornecer cuidados médicos), cada um em um programa diferente (receita *versus* livro de primeiros socorros). Quando a picada de abelha tiver sido tratada, o cientista voltará ao seu bolo, continuando do ponto em que parou.

A ideia principal é que um processo constitui uma atividade. Ele possui programa, entrada, saída e um estado. Um único processador pode ser compartilhado entre os vários processos, com algum algoritmo de escalonamento usado para determinar quando parar o trabalho sobre um processo e servir outro.

Convém notar que, se um programa está sendo executado duas vezes, isso conta como dois processos. Por exemplo, frequentemente é possível iniciar um processador de texto duas vezes ou imprimir dois arquivos ao mesmo tempo, se duas impressoras estiverem disponíveis.

O fato de que dois processos em execução estejam operando o mesmo programa não importa; eles são processos diferentes. O sistema operacional pode compartilhar o código entre eles e, desse modo, apenas uma cópia está na memória, mas esse é um detalhe técnico que não altera a situação conceitual dos dois processos sendo executados.

7.2 Criação de processos

Os sistemas operacionais precisam de mecanismos para criar processos. Em sistemas muito simples, ou em sistemas projetados para executar uma única aplicação (por exemplo, o controlador do forno de micro-ondas), pode ser possível que todos os processos que serão necessários sejam criados quando o sistema é ligado. Contudo, em sistemas de propósito geral, é necessário algum mecanismo para criar e terminar processos durante a operação, quando for preciso. Veremos agora alguns desses tópicos.

Há quatro eventos principais que fazem com que processos sejam criados:

1. Início do sistema.
2. Execução de uma chamada de sistema de criação de processo por um processo em execução.
3. Uma requisição do usuário para criar um novo processo.
4. Início de uma tarefa em lote (*batch job*).

Quando um sistema operacional é carregado, em geral, criam-se vários processos. Alguns deles são processos em *foreground* (primeiro plano), ou seja, que interagem com usuários (humanos) e realizam tarefas para eles. Outros são processos em *background* (segundo plano), que não estão associados a usuários em particular, mas que apresentam alguma função específica.

Por exemplo, um processo em *background* (segundo plano) pode ser designado a aceitar mensagens eletrônicas sendo recebidas, ficando inativo na maior parte do dia, mas surgindo de repente quando uma mensagem chega.

Outro processo em *background* (segundo plano) pode ser destinado a aceitar solicitações que chegam para páginas da *web* hospedadas naquela máquina, despertando quando uma requisição chega pedindo o serviço.

Processos que ficam em *background* com a finalidade de lidar com alguma atividade, como mensagem eletrônica, páginas da *web*, notícias, impressão, entre outros, são chamados de *daemons*.

É bem comum sistemas lançarem mão de dezenas deles. No UNIX, o programa `ps` pode ser usado para relacionar os processos que estão executando. No Windows, o gerenciador de tarefas pode ser usado.

Além dos processos criados durante a carga do sistema operacional, novos processos podem ser criados depois disso. Muitas vezes, um processo em execução fará chamadas de sistema (*system calls*) para criar um ou mais novos processos para ajudá-lo em seu trabalho.

Criar novos processos é particularmente interessante, quando a tarefa a ser executada puder ser facilmente dividida em vários processos relacionados, mas interagindo de maneira independente.

Por exemplo, se uma grande quantidade de dados estiver sendo trazida via rede para que seja subsequentemente processada, poderá ser conveniente criar um processo para trazer esses dados e armazená-los em um local compartilhado da memória, enquanto um segundo processo remove os dados e os processa. Em um sistema multiprocessador, permitir que cada processo execute em uma CPU diferente também torna o trabalho mais rápido.

Em sistemas interativos, os usuários podem inicializar um programa digitando um comando ou clicando (duas vezes) um ícone. Cada uma dessas ações inicia um novo processo e executa nele o programa selecionado. Em sistemas UNIX baseados em comandos que executam o X, o novo processo toma posse da janela na qual ele foi disparado.

No Microsoft Windows, quando um processo é disparado, ele não tem uma janela, mas pode criar uma (ou mais de uma), e a maioria deles cria. Nos dois sistemas, os usuários podem ter múltiplas janelas abertas ao mesmo tempo, cada uma executando algum processo. Usando o mouse, o usuário seleciona uma janela e interage com o processo, por exemplo, fornecendo a entrada de dados quando for solicitado.

A última situação na qual processos são criados aplica-se somente a sistemas em lote encontrados em computadores de grande porte. Nesses sistemas, usuários podem submeter (até remotamente) tarefas em lote para o sistema. Quando julgar que tem recursos para executar outra tarefa, o sistema operacional criará um novo processo e executará nele a próxima tarefa da fila de entrada.

Tecnicamente, em todos esses casos, um novo processo (processo filho) é criado por um processo existente (processo pai), executando uma chamada de sistema para a sua criação. Esse processo (pai) pode ser um processo de usuário que está executando, um processo de sistema invocado a partir do teclado ou do mouse ou um processo gerenciador de lotes. O que ele faz é executar uma chamada de sistema para criar um novo processo (filho) e assim indica, direta ou indiretamente, qual programa executar nele.

No UNIX, há somente uma chamada de sistema para criar um novo processo: *fork*. Essa chamada cria um clone idêntico ao processo que a chamou. Depois da *fork*, os dois processos, o pai e o filho, têm a mesma imagem de memória, as mesmas variáveis de ambiente e os mesmos arquivos abertos. E isso é tudo.

Normalmente, o processo filho executa, em seguida, uma chamada de sistema similar para mudar sua imagem de memória e executar um novo programa. Por exemplo, quando um usuário digita um comando *sort* no interpretador de comandos, ele se bifurca, gerando um processo filho, e o processo filho executa o *sort*.

A razão para esse processo de dois passos é permitir que o filho manipule seus descritores de arquivos depois da *fork*, mas antes da chamada de sistema, para conseguir redirecionar a entrada-padrão, a saída-padrão e a saída de erros-padrão.

Por outro lado, no Windows, uma única chamada de função do Win32, **CreateProcess**, trata tanto do processo de criação quanto da carga do programa correto no novo processo.

Essa chamada possui dez parâmetros, incluindo o programa a ser executado, os parâmetros da linha de comando que alimentam esse programa, vários atributos de segurança, os *bits* que controlam se os arquivos abertos são herdados, informação sobre prioridade, uma especificação da janela a ser criada para o processo (se houver) e um ponteiro para uma estrutura na qual a informação sobre o processo recém-criado é retornada para quem chamou.

Além do **CreateProcess**, o Win32 apresenta cerca de cem outras funções para gerenciar e sincronizar processos e tópicos afins.

Tanto no UNIX quanto no Windows, depois que um processo é criado, o pai e o filho têm seus próprios espaços de endereçamento distintos. Se um dos dois processos alterar uma palavra em seu espaço de endereçamento, a mudança não será visível ao outro processo.

No UNIX, o espaço de endereçamento inicial do filho é uma cópia do espaço de endereçamento do pai, mas há dois espaços de endereçamento distintos envolvidos; nenhuma memória para escrita é compartilhada (algumas implementações UNIX compartilham o código do programa entre os dois, já que não podem ser alteradas).

Contudo, é possível que um processo recentemente criado compartilhe algum de seus recursos com o processo que o criou, como arquivos abertos. No Windows, os espaços de endereçamento do pai e do filho são diferentes desde o início.

7.3 Término de processos

Depois de criado, um processo começa a executar e faz seu trabalho. Contudo, nada é para sempre, nem mesmo os processos. Mais cedo ou mais tarde o novo processo terminará, normalmente em razão de alguma das seguintes condições:

1. Saída normal (voluntária).
2. Saída por erro (voluntária).
3. Erro fatal (involuntário).
4. Cancelamento por outro processo (involuntário).

Na maioria das vezes, os processos terminam porque fizeram seu trabalho. Quando acaba de compilar o programa atribuído a ele, o compilador executa uma chamada de sistema para dizer ao sistema operacional que ele terminou. Essa chamada é a `exit` no UNIX e a `ExitProcess` no Windows.

Programas baseados em tela também suportam o término voluntário. Processadores de texto, visualizadores da Web (*browsers*) e programas similares sempre têm um ícone ou um item de menu no qual o usuário pode clicar para dizer ao processo que remova quaisquer arquivos temporários que ele tenha aberto e, então, termine.

O segundo motivo para término é que o processo descobre um erro fatal. Por exemplo, se um usuário digita o comando `cc foo.c` para compilar o programa `foo.c` e esse arquivo não existe, o compilador simplesmente termina a execução. Processos interativos com base em tela geralmente não fecham quando parâmetros errados são fornecidos. Em vez disso, uma caixa de diálogo emerge e pergunta ao usuário se ele quer tentar novamente.

A terceira razão para o término é um erro causado pelo processo, muitas vezes por um erro de programa. Entre os vários exemplos, estão a execução de uma instrução ilegal, a referência à memória inexistente ou a divisão por zero. Em alguns sistemas (por exemplo, UNIX), um processo pode dizer ao sistema operacional que deseja, ele mesmo, tratar certos erros. Nesse caso, o processo é sinalizado (interrompido) em vez de finalizado pela ocorrência de erros.

A quarta razão pela qual um processo pode terminar se dá quando um processo executa uma chamada de sistema dizendo ao sistema operacional para cancelar algum outro processo. No UNIX, essa chamada é a `kill`. A função Win32 correspondente é a `TerminateProcess`.

Em ambos os casos, o processo que for efetuar o cancelamento deve ter a autorização necessária para fazê-lo. Em alguns sistemas, quando um processo termina, voluntariamente ou não, todos os processos criados por ele também são imediatamente cancelados. Contudo, nem o UNIX nem o Windows funcionam dessa maneira.

7.4 Hierarquias de processos

Em alguns sistemas, quando um processo cria outro, o processo pai e o processo filho continuam, de certa maneira, associados. O próprio processo filho pode gerar mais processos, formando uma hierarquia. Observe que isso é diferente do que ocorre com plantas e animais, que utilizam a reprodução sexuada, pois um processo tem apenas um pai (mas pode ter nenhum, um, dois ou mais filhos).

No UNIX, um processo, todos os seus filhos e descendentes formam um grupo de processos. Quando um usuário envia um sinal do teclado, o sinal é entregue a todos os membros do grupo de processos associado com o teclado (normalmente, todos os processos ativos que foram criados na janela atual). Individualmente, cada processo pode capturar o sinal, ignorá-lo ou tomar a ação predefinida, isto é, ser finalizado pelo sinal.

Outro exemplo da atuação dessa hierarquia pode ser observado no início do UNIX, quando o computador é ligado. Um processo especial, chamado `init`, está presente na imagem de carga do sistema. Quando começa a executar, ele lê um arquivo dizendo quantos terminais existem. Então, ele se bifurca várias vezes para ter um novo processo para cada terminal. Esses processos esperam por alguma conexão de usuário. Se algum usuário se conectar, o processo de conexão executará um interpretador de comandos para aceitar comandos dos usuários. Esses comandos podem iniciar mais processos, e assim por diante. Desse modo, todos os processos em todo o sistema pertencem a uma única árvore, com o `init` na raiz.

Por outro lado, o Windows não apresenta nenhum conceito de hierarquia de processos. Todos são iguais. Algo parecido com uma hierarquia de processos ocorre somente quando um processo é criado. Ao pai é dado um identificador especial (chamado `handle`), que ele pode usar para controlar o filho. Contudo, ele é livre para passar esse identificador para alguns outros processos, invalidando, assim, a hierarquia. Os processos no UNIX não podem deserdar seus filhos.

7.5 Estados de processos

Embora cada processo seja uma entidade independente, com seu próprio contador de programa e estado interno, muitas vezes os processos precisam interagir com outros. Um processo pode gerar uma saída que outro processo usa como entrada.

```
$ cat chapter1 chapter2 chapter3 | grep tree
```

Comando de prompt 10.1 – Comando cat e comando grep
Fonte: Elaborado pelo autor (2019)

Veja o exemplo apresentado no comando de prompt “Comando `cat` e comando `grep`”: o primeiro processo, que executa `cat`, gera como saída a concatenação dos três arquivos. O segundo processo, que executa `grep`, seleciona todas as linhas contendo a palavra 'tree'.

Dependendo das velocidades relativas dos dois processos (atreladas tanto à complexidade relativa dos programas quanto ao tempo de CPU que cada um deteve), pode ocorrer que o `grep` esteja pronto para executar, mas não haja entrada para ele. Ele deve, então, bloquear até que alguma entrada esteja disponível.

Um processo bloqueia porque, obviamente, não pode prosseguir em geral visto que está esperando por uma entrada ainda não disponível. É possível também que um processo conceitualmente pronto e capaz de executar esteja bloqueado porque o sistema operacional decidiu alocar a CPU para outro processo por algum tempo.

Essas duas condições são completamente diferentes. No primeiro caso, a suspensão é inerente ao problema (não se pode processar a linha de comando do usuário enquanto ele não digitar nada). O segundo é uma técnica do sistema (não há CPUs suficientes para dar a cada processo um processador exclusivo). Na Figura “Estados de um processo”, podemos ver um diagrama de estados mostrando os três estados de um processo:

1. Em execução (realmente usando a CPU naquele instante).
2. Pronto (executável; temporariamente parado para dar lugar a outro processo).
3. Bloqueado (incapaz de executar enquanto não ocorrer um evento externo).

Logicamente, os dois primeiros estados são similares. Em ambos os casos, o processo vai executar, só que no segundo não há, temporariamente, CPU disponível para ele. O terceiro estado é diferente dos dois primeiros, pois o processo não pode executar, mesmo que a CPU não tenha nada para fazer.

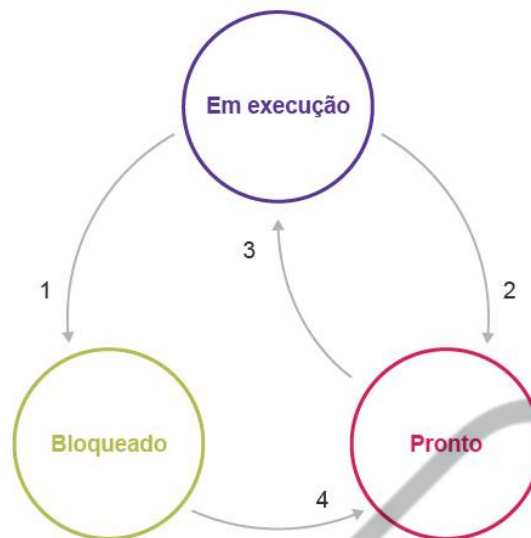


Figura 18 – Estados de um processo
Fonte: TANENBAUM (2009)

1. O processo bloqueia aguardando uma entrada;
2. O escalonador seleciona outro processo;
3. O escalonador seleciona esse processo;
4. A entrada torna-se disponível.

Um processo pode estar nos estados em execução, bloqueado ou pronto. As transições entre esses estados são mostradas na Figura “Estados de um processo”.

Quatro transições são possíveis entre esses três estados, conforme se vê na Figura “Estados de um processo”. A transição 1 ocorre quando o sistema operacional descobre que um processo não pode prosseguir. Em alguns sistemas, o processo precisa executar uma chamada de sistema, como pause, para entrar no estado bloqueado.

Em outros sistemas, inclusive no UNIX, quando um processo lê de um pipe ou de um arquivo especial (por exemplo, um terminal) e não há entrada disponível, o processo é automaticamente bloqueado.

As transições 2 e 3 são causadas pelo escalonador de processos — uma parte do sistema operacional, sem que o processo saiba disso. A transição 2 ocorre quando o escalonador decide que o processo em execução já teve tempo suficiente de CPU e é momento de deixar outro processo ocupar o tempo da CPU.

A transição 3 ocorre quando todos os outros processos já compartilharam a CPU, de uma maneira justa, e é hora de o primeiro processo obter novamente a CPU.

O escalonamento, isto é, a decisão sobre quando e por quanto tempo cada processo deve executar, é um tópico muito importante e será estudado depois, neste mesmo capítulo. Muitos algoritmos vêm sendo desenvolvidos na tentativa de equilibrar essa competição, que exige eficiência para o sistema como um todo e igualdade para os processos individuais.

A transição 4 ocorre quando acontece um evento externo pelo qual um processo estava aguardando (como a chegada de alguma entrada). Se nenhum outro processo estiver executando naquele momento, a transição 3 será disparada e o processo começará a executar. Caso contrário, ele poderá ter de aguardar em estado de pronto por um pequeno intervalo de tempo, até que a CPU esteja disponível e chegue sua vez.

Com o modelo de processo, torna-se muito mais fácil saber o que está ocorrendo dentro do sistema. Alguns dos processos chamam programas que executam comandos digitados por um usuário. Outros processos são parte do sistema e manejam tarefas como fazer requisições por serviços de arquivos ou gerenciar os detalhes do funcionamento de um acionador de disco ou fita.

Quando ocorre uma interrupção de disco, o sistema toma a decisão de parar de executar o processo corrente e retomar o processo do disco que foi bloqueado para aguardar essa interrupção. Assim, em vez de pensar em interrupções, podemos pensar em processos de usuário, de disco, de terminais ou outros, que bloqueiam quando estão à espera de que algo aconteça. Finalizada a leitura do disco ou a digitação de um caractere, o processo que aguarda por isso é desbloqueado e torna-se disponível para executar novamente.

8 ESCALONAMENTO

De volta aos velhos tempos dos sistemas em lote, com a entrada na forma de imagens de cartões em uma fita magnética, o algoritmo de escalonamento era simples: apenas execute a próxima tarefa que está na fita.

Com os sistemas multiprogramados, o algoritmo de escalonamento tornou-se mais complexo porque, em geral, havia vários usuários esperando por um serviço. Alguns computadores de grande porte ainda combinam serviços em lote e de tempo compartilhado, exigindo, assim, que o escalonador decida se uma tarefa em lote ou um usuário interativo em um terminal deve ser atendido. Atente para o seguinte: uma tarefa em lote pode ser uma requisição para executar uma sucessão de vários programas, mas, para essa seção, vamos supor que seja uma requisição para executar um único programa.

Como o tempo de CPU é um recurso escasso nessas máquinas, um bom escalonador pode fazer uma grande diferença no desempenho observado e na satisfação do usuário. Consequentemente, muito se fez tendo em vista desenvolver algoritmos de escalonamento inteligentes e eficientes.

Com o advento dos computadores pessoais, a situação mudou de duas maneiras. Primeiro, na maior parte do tempo existe apenas um processo ativo. É improvável que um usuário esteja, simultaneamente, entrando com um documento em um processador de textos e compilando um programa em segundo plano. Quando o usuário digita um comando para o processador de textos, o escalonador não precisa trabalhar muito para perceber qual processo executar: o processador de textos é o único candidato.

Em segundo lugar, os computadores, com o passar dos anos, ficaram tão mais rápidos que a CPU raramente chegará a ser um recurso escasso. A maioria dos programas para computadores pessoais é limitada pela velocidade com que o usuário pode entrar dados (digitando ou clicando), e não pela taxa na qual a CPU é capaz de processá-los.

Até os compiladores, grandes consumidores de ciclos de CPU no passado, atualmente levam, no máximo, alguns segundos. Mesmo quando dois programas estiverem executando de modo simultâneo como um processador de textos e uma planilha, dificilmente importará qual começa primeiro, já que o usuário estará esperando, provavelmente, que ambos terminem. Como consequência, o escalonamento não é tão importante em PCs simples.

Claro, há aplicações que praticamente esgotam a CPU: exibir uma hora de vídeo de alta resolução pode exigir processamento de imagens de altíssima capacidade para lidar com cada um dos 108 mil quadros em NTSC (90 mil no PAL), mas essas aplicações são exceções, não a regra.

Quando nos concentramos em servidores e estações de trabalho de alto desempenho em rede, a situação muda. Nesse caso, é comum haver múltiplos processos competindo pela CPU, e, portanto, o escalonamento torna-se importante novamente. Por exemplo, quando a CPU precisar decidir entre executar um processo que reúne estatísticas diárias e um que atende às solicitações dos usuários, eles ficarão muito mais satisfeitos se o último tiver precedência na CPU.

Além de escolher o processo certo para executar, o escalonador também deve se preocupar em fazer um uso eficiente da CPU, pois chavear processos é muito custoso.

De início, deve ocorrer um chaveamento do modo de usuário para o modo núcleo. Depois, o estado atual do processo deve ser salvo, armazenando-se, inclusive, seus registradores na tabela de processos, para que possam ser recarregados posteriormente.

Em muitos sistemas, o mapa de memória (por exemplo, os bits de referência à memória na tabela de páginas) também deve ser salvo. Em seguida, um novo processo precisa ser selecionado pela execução do algoritmo de escalonamento.

Depois disso, a MMU (*memory management unit* — unidade de gerenciamento de memória) tem de ser recarregada com o mapa de memória do novo processo. Por fim, o novo processo precisa ser iniciado.

Além disso tudo, o chaveamento do processo normalmente invalida toda a memória cache, forçando-a a ser dinamicamente recarregada da memória principal por duas vezes (ao entrar no núcleo e ao sair dele). De modo geral, realizar muitos chaveamentos de processos por segundo pode comprometer uma grande quantidade do tempo de CPU; portanto, todo cuidado é pouco.

8.1 Comportamento do processo

Quase todos os processos alternam surtos de computação com requisições de E/S (de disco), conforme mostra a Figura “Comportamento de um processo”. Em geral, a CPU executa indefinidamente e então é feita uma chamada de sistema para ler de um arquivo ou escrever nele. Quando a chamada de sistema termina, a CPU computa novamente até que ela requisite ou precise escrever mais dados, e assim continua.

Perceba que algumas atividades de E/S contam como computação. Por exemplo, quando a CPU copia os *bits* para uma RAM de vídeo a fim de atualizar a tela, ela está computando, não fazendo E/S, pois a CPU se encontra em uso. E/S, nesse sentido, é o que ocorre quando um processo entra no estado bloqueado, esperando que um dispositivo externo termine o que está fazendo.

O que é importante observar na Figura “Comportamento de um processo” é que alguns processos, como em (a), gastam a maior parte do tempo computando, enquanto outros, como em (b), passam a maior parte de seu tempo esperando E/S. Os primeiros são chamados limitados pela CPU (*compute-bound* ou *CPU-bound*); os últimos são os limitados pela E/S (*I/O-bound*).

Os processos limitados pela CPU apresentam, em geral, longos surtos de uso da CPU e esporádicas esperas por E/S; já os processos limitados por E/S têm pequenos surtos de uso da CPU e esperas frequentes por E/S.

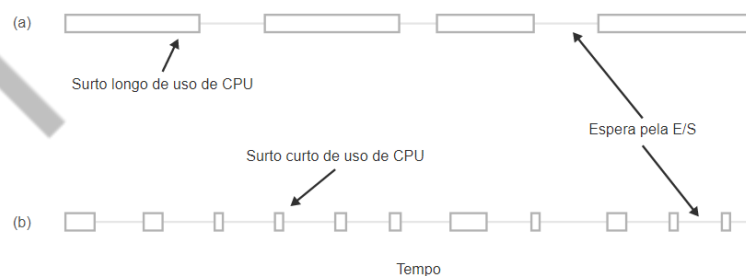


Figura 19 – Comportamento de um processo
Fonte: TANENBAUM (2009)

Note que o fator principal é o tamanho do surto de CPU, não o tamanho do surto de E/S. Os processos orientados à E/S são assim chamados porque, entre uma requisição e outra por E/S, eles não realizam muita computação, não porque tenham requisições por E/S especialmente demoradas. O tempo para a leitura de um bloco de disco é sempre o mesmo, independentemente do quanto demore processar os dados que chegam depois.

Convém observar que, à medida que as CPUs se tornam mais rápidas, os processos tendem a ficar mais limitados por E/S. Esse efeito ocorre porque as CPUs estão ficando muito mais rápidas que os discos. Como consequência, o escalonamento de processos orientados à E/S deverá ser um assunto mais importante no futuro. A ideia básica é que, se um processo orientado à E/S quiser executar, essa oportunidade deve ser rapidamente dada a ele, pois assim ele executará suas requisições de disco, mantendo o disco ocupado.

8.2 Quando escalonar

Um tópico fundamental, relacionado ao escalonamento, é o momento certo de tomar as decisões de escalonar. É claro que há uma variedade de situações nas quais o escalonamento é necessário. Primeiro, quando se cria um novo processo, é necessário tomar uma decisão entre executar o processo pai ou o processo filho.

Como ambos os processos estão no estado pronto, essa é uma decisão normal de escalonamento e pode levar à escolha de um ou de outro, isto é, o escalonador pode escolher legitimamente executar o pai ou o filho.

Em segundo lugar, uma decisão de escalonamento deve ser tomada ao término de um processo. Como esse não pode executar mais (já que ele não existe mais), algum outro processo deve ser escolhido entre os processos prontos. Se nenhum estiver pronto, é executado um processo ocioso gerado pelo sistema.

Em terceiro lugar, quando um processo bloqueia para E/S, sobre um semáforo ou por alguma outra razão, outro processo precisa ser selecionado para executar. O motivo do bloqueio pode, algumas vezes, influenciar na escolha. Por exemplo, se A for um processo importante e estiver esperando B sair de sua região crítica, deixar B executar em seguida permitirá que ele saia de sua região crítica e, portanto, permite que A continue. O problema, contudo, é que geralmente o escalonador não possui a informação necessária para considerar essa dependência.

Em quarto lugar, quando ocorre uma interrupção de E/S, pode-se tomar uma decisão de escalonamento. Se a interrupção vem de um dispositivo de E/S que acabou de fazer seu trabalho, o processo que estava bloqueado, esperando pela E/S, pode agora ficar pronto para execução. É o escalonador quem decide se executa o processo que acabou de ficar pronto, o processo que estava executando no momento da interrupção ou algum terceiro processo.

Se um hardware de relógio fornece interrupções periódicas a 50 Hz, 60 Hz ou alguma outra frequência, uma decisão de escalonamento pode ser tomada a cada interrupção de relógio ou a cada k-ésima interrupção de relógio.

Os algoritmos de escalonamento podem ser divididos em duas categorias quanto ao modo como tratam essas interrupções. Um algoritmo de escalonamento não *preemptivo* escolhe um processo para executar e, então, o deixa executar até que seja bloqueado (à espera de E/S ou de outro processo) ou até que ele voluntariamente libere a CPU.

Mesmo que ele execute por horas, não será compulsoriamente suspenso. Na verdade, nenhuma decisão de escalonamento é tomada durante as interrupções de relógio. Depois que o processamento da interrupção de relógio termina, o processo que estava executando antes da interrupção prossegue até acabar, a menos que um processo de prioridade mais alta esteja esperando por um tempo de espera agora satisfeito.

Por outro lado, um algoritmo de escalonamento preemptivo escolhe um processo e o deixa em execução por um tempo máximo fixado. Se ainda estiver executando ao final desse intervalo de tempo, o processo será suspenso e o escalonador escolherá outro processo para executar (se houver algum disponível).

O escalonamento preemptivo requer a existência de uma interrupção de relógio ao fim do intervalo de tempo para que o controle sobre a CPU seja devolvido ao escalonador. Se não houver relógio disponível, o escalonamento não preemptivo será a única opção.

8.3 Categorias de algoritmos de escalonamento

É claro que, para ambientes diferentes, são necessários diferentes algoritmos de escalonamento. Essa situação ocorre porque diferentes áreas de aplicação (e diferentes tipos de sistemas operacionais) têm objetivos diferentes. Em outras palavras, o que deve ser otimizado pelo escalonador não é o mesmo para todos os sistemas. Três ambientes merecem distinção:

1. Lote.
2. Interativo.
3. Tempo real.

Os sistemas em lote ainda são amplamente utilizados pelas empresas para folhas de pagamento, estoque, contas a receber, contas a pagar, cálculo de juros (em bancos), processamento de pedidos de indenização (em companhias de seguros) e outras tarefas periódicas.

Nos sistemas em lote não há, em seus terminais, usuários esperando impacientes por uma resposta rápida. Consequentemente, os algoritmos não preemptivos ou preemptivos com longo intervalo de tempo para cada processo são, em geral, aceitáveis.

Essa tática reduz os chaveamentos entre processos e assim melhora o desempenho. Na verdade, os algoritmos de lote são bastante comuns e muitas vezes aplicáveis a outras situações também, o que torna importante estudá-los, até para pessoas que não estejam envolvidas em computação central corporativa.

Em um ambiente com usuários interativos, a preempção é essencial para evitar que um processo se aposse da CPU e, com isso, negue serviço aos outros. Mesmo que nenhum processo execute intencionalmente para sempre, uma falha em um programa pode levar um processo a impedir indefinidamente que todos os outros executem. A preempção é necessária para impedir esse comportamento. Os servidores também caem nessa categoria, visto que normalmente servem a usuários (remotos) múltiplos, todos muito apressados.

Em sistemas com restrições de tempo real, a preempção é, estranhamente, algumas vezes desnecessária, pois os processos sabem que não podem executar por longos períodos e, em geral, fazem seus trabalhos e bloqueiam rapidamente.

A diferença com relação aos sistemas interativos é que os sistemas de tempo real executam apenas programas que visam ao progresso da aplicação. Já os sistemas interativos são de propósito geral e podem executar programas arbitrários não cooperativos ou até mal-intencionados.

8.4 Objetivos do algoritmo de escalonamento

Para projetar um algoritmo de escalonamento, é necessário ter alguma ideia do que um bom algoritmo deve fazer. Alguns objetivos dependem do ambiente (lote, interativo ou tempo real), mas há também aqueles que são desejáveis para todos os casos. Alguns objetivos são relacionados na Tabela 1.

Em qualquer circunstância, justiça é algo importante. Processos semelhantes devem ter serviços semelhantes. Não é justo dar mais tempo de CPU a um processo do que a outro equivalente. É claro que categorias diferentes de processos podem ser tratadas de modo muito diverso. Pense no controle de segurança e na folha de pagamento de um centro de computação de uma usina nuclear.

De alguma maneira relacionada à justiça está o cumprimento das políticas do sistema. Se a política local estabelece que os processos do controle de segurança executem quando quiserem, mesmo que isso signifique que a folha de pagamento atrase 30 segundos, o escalonador deve assegurar que essa política seja cumprida.

Todos os sistemas	
Justiça	dar a cada processo uma porção justa da CPU
Aplicação da política	verificar se a política estabelecida é cumprida com Equilíbrio — manter ocupadas todas as partes do sistema
Sistemas em lote	
Vazão (<i>throughput</i>)	maximizar o número de tarefas por hora
Tempo de retorno	minimizar o tempo entre a submissão e o término
Utilização de CPU	manter a CPU ocupada o tempo todo
Sistemas interativos	
Tempo de resposta	responder rapidamente às requisições
Proporcionalidade	satisfazer às expectativas dos usuários
Sistemas de tempo real	
Cumprimento dos prazos	evitar a perda de dados
Previsibilidade	evitar a degradação da qualidade em sistemas multimídia

Quadro 1 – Alguns objetivos do algoritmo de escalonamento sob diferentes circunstâncias.

Fonte: Elaborado pelo autor (2019)

Outro objetivo geral é manter, quando possível, todas as partes do sistema ocupadas. Se a CPU e os demais dispositivos de E/S puderem ser mantidos em execução o tempo todo, mais trabalho por segundo será feito do que se algum dos componentes estiver ocioso.

Em um sistema com lote, por exemplo, o escalonador tem o controle de quais tarefas são trazidas para a memória para executar. É melhor ter juntos, na memória, alguns processos limitados pela CPU e outros limitados por E/S do que carregar todas as tarefas limitadas pela CPU primeiro e quando terminarem, carregar e executar todas as tarefas limitadas por E/S.

Se a última estratégia for usada, quando os processos orientados à CPU estiverem executando, disputarão a CPU e, assim, o disco ficará ocioso. Em seguida, quando as tarefas limitadas por E/S executarem, disputarão o disco e a CPU se encontrará ociosa. É melhor manter o sistema todo executando de uma vez, formulando-se cuidadosamente essa mistura de processos.

Os gerentes de grandes centros de computação que executam muitas tarefas em lote observam, em geral, três métricas para verificar se os sistemas deles estão executando bem ou não: vazão, tempo de retorno e utilização da CPU.

Vazão é o número de tarefas por hora que o sistema termina. Considerando tudo o que foi discutido, terminar 50 tarefas por hora é melhor do que terminar 40 no mesmo período. O tempo de retorno é estatisticamente o tempo médio do momento em que uma tarefa em lote é submetida até o momento em que ela é terminada. Ele indica quanto tempo, em média, o usuário tem de esperar pelo fim de um trabalho. Aqui, a regra é: quanto menor, melhor.

Um algoritmo de escalonamento que maximize a vazão pode não necessariamente minimizar o tempo de retorno. Por exemplo, dada uma mistura de tarefas curtas e longas, um escalonador que execute sempre tarefas curtas e nunca tarefas longas pode conseguir uma excelente vazão (muitas tarefas curtas por hora), mas à custa de um enorme tempo de retorno para as tarefas longas. Se as tarefas curtas mantiverem uma taxa de chegada constante, as tarefas longas poderão nunca executar, tornando o tempo médio de retorno infinito, embora atingindo alta vazão.

A utilização da CPU também é muitas vezes usada como parâmetro em sistemas em lote. Na verdade, esse não é um bom parâmetro. O que realmente interessa é quantas tarefas por hora saem do sistema (vazão) e quanto tempo leva para receber o resultado do trabalho (tempo de retorno).

Tomar a utilização da CPU como medida é o mesmo que avaliar um carro pelo número de giros que seu motor dá a cada hora. Por outro lado, saber quando a utilização da CPU está se aproximando de 100 por cento é útil para identificar o momento de obter mais potência para o computador.

Para sistemas interativos, aplicam-se objetivos diferentes. O que importa é minimizar o tempo de resposta, isto é, o tempo entre a emissão de um comando e a obtenção do resultado.

Em um computador pessoal, no qual está sendo executado um processo em segundo plano (por exemplo, lendo e armazenando mensagens de correio eletrônico da rede), uma requisição de usuário, para inicializar um programa ou abrir um arquivo, deveria ter precedência sobre o trabalho em segundo plano. Atender antes a todas as requisições interativas será considerado um bom serviço.

Uma questão relacionada a esse tópico é a chamada proporcionalidade. Os usuários têm uma intuição (mas muitas vezes errada) de quanto tempo as coisas devem durar. Quando uma requisição tida como complexa demora, os usuários aceitam isso, mas, quando a demora ocorre com uma requisição considerada simples, os usuários ficam irritados.

Por exemplo, se, ao clicar em um ícone que inicia o envio de um fax, são necessários 60 segundos para concluí-lo, o usuário provavelmente encarará isso como inevitável, porque não espera que um fax seja enviado em cinco segundos.

Por outro lado, quando o usuário clicar em um ícone para interromper a conexão telefônica após o envio do fax, ele tem expectativas diferentes. Se não tiver sido concluído após 30 segundos, o usuário provavelmente começará a reclamar e, após 60 segundos, estará espumando de raiva.

Esse comportamento é causado pela inevitável comparação que o usuário faz, de que realizar uma chamada telefônica e passar um fax deveria demorar muito mais do que desligar o telefone. Em alguns casos (como esse), o escalonador não pode fazer nada com relação ao tempo de resposta, mas em outros casos sim, especialmente naqueles em que o atraso é decorrente de uma má escolha da ordem dos processos.

Sistemas de tempo real têm propriedades diferentes dos sistemas interativos e, portanto, objetivos diferentes. Eles são caracterizados por prazos que devem, ou pelo menos deveriam, ser cumpridos. Por exemplo, em um computador encarregado de controlar um dispositivo que produza dados a uma taxa constante, uma falha ao executar o processo de coleta de dados em tempo hábil pode resultar na perda de dados. Assim, a principal exigência de um sistema de tempo real é cumprir todos os prazos (ou a maior parte deles).

Em alguns sistemas de tempo real, especialmente naqueles que envolvem multimídia, a previsibilidade é importante. Deixar de cumprir um prazo ocasional não é fatal, mas, se o processo de áudio, por exemplo, executar erráticamente, a qualidade do som vai deteriorar rápido. O vídeo também é um problema, mas o ouvido é muito mais sensível a atrasos que a visão. Para evitar esse problema, o escalonamento de processos deve ser altamente previsível e regular.

9 GERENCIAMENTO DE MEMÓRIA

A memória principal (RAM) é um recurso importante que deve ser gerenciado com muito cuidado. Apesar de, atualmente, os computadores pessoais possuírem memórias dez mil vezes maiores que o IBM 7094 (o maior computador do mundo no início dos anos 1960), os programas tornam-se maiores muito mais rapidamente do que as memórias.

Parafraseando a Lei de Parkinson, pode-se afirmar que "programas tendem a se expandir a fim de ocupar toda a memória disponível". Estudaremos a seguir como os sistemas operacionais criam abstrações a partir da memória e como as gerenciam.

O que todo programador deseja é dispor de uma memória infinitamente grande, rápida e não volátil, ou seja, uma memória que não perdesse seu conteúdo quando faltasse energia. E por que não também a um baixo custo? Infelizmente, a tecnologia atual não comporta essas memórias. Talvez você seja capaz de desenvolvê-las.

Qual é a segunda opção? Ao longo dos anos, as pessoas descobriram o conceito de hierarquia de memórias, em que os computadores têm alguns megabytes de memória cache muito rápida, de custo alto e volátil, alguns gigabytes de memória principal volátil de velocidade e custo médios e alguns terabytes de armazenagem em disco não volátil de velocidade e custo baixos, para não falar do armazenamento removível, como DVDs e dispositivos USB. A função do sistema operacional é abstrair essa hierarquia em um modelo útil e, então, gerenciar a abstração.

A parte do sistema operacional que gerencia (parcialmente) a hierarquia de memórias é denominada gerenciador de memória. Sua função é gerenciar a memória de modo eficiente: manter o controle sobre quais partes da memória estão em uso e quais não estão alocando memória aos processos quando eles precisam e liberando quando esses processos terminam.

Neste capítulo, conheceremos vários esquemas diferentes de gerenciamento de memória, desde os mais simples aos mais sofisticados. Visto que o gerenciamento do nível inferior da memória cache normalmente é feito pelo hardware, o foco deste capítulo será o modelo da memória principal do programador e como pode ser bem gerenciado.

As abstrações para armazenamento permanente, o disco e o gerenciamento dessas abstrações são o tema do próximo capítulo. Começaremos pelo sistema mais simples possível e, então, avançaremos gradualmente para os mais elaborados.

9.1 Sem abstração de memória

A abstração de memória mais simples é a ausência de abstração. Os primeiros computadores de grande porte (antes de 1960), microcomputadores (antes de 1970) e computadores pessoais (antes de 1980) não possuíam abstração de memória. Cada programa simplesmente considerava a memória física. Quando um programa executava uma instrução como:

MOV REGISTER1,1000

O computador apenas movia o conteúdo da memória física da posição 1000 para RF.GISTF.R1. Assim, o modelo de memória apresentado ao programador era simplesmente a memória física, um conjunto de endereços de 0 a algum máximo, cada endereço correspondendo a uma célula que continha certo número de bits, normalmente oito.

Nessas condições, não era possível executar dois programas na memória, simultaneamente. Se o primeiro programa escrevesse um novo valor para a posição 2000, por exemplo, apagaria qualquer valor que o segundo programa estivesse armazenando ali. Nenhum deles funcionaria e os dois programas quebrariam quase imediatamente.

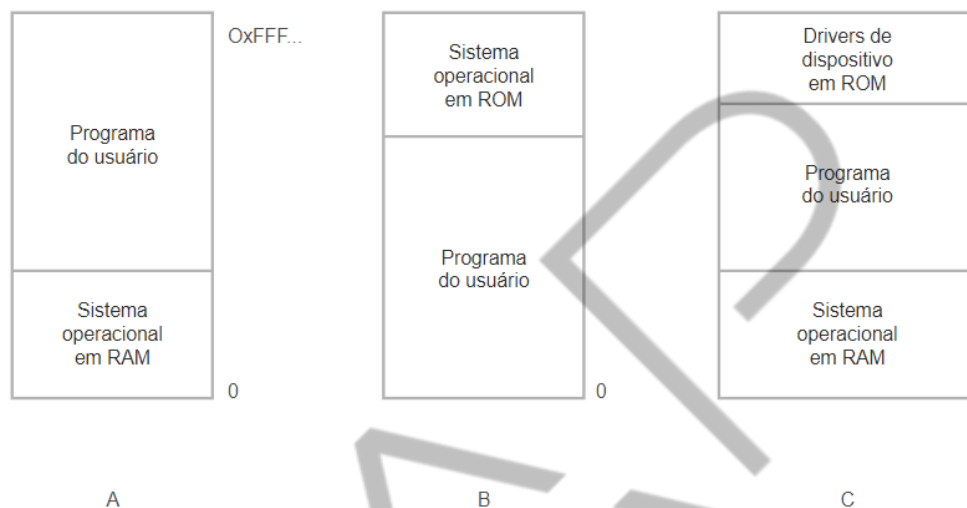


Figura 20 – Três modos simples de organizar a memória
Fonte: TANENBAUM (2009)

Ainda que o modelo da memória fosse apenas a memória física, havia várias opções possíveis. São mostradas três variações na Figura “Três modos simples de organizar a memória”.

O sistema operacional pode estar na parte inferior da memória em RAM (*random access memory*, memória de acesso aleatório), como mostrado em (a), ou pode estar em ROM (*read-only memory* — memória apenas para leitura) na parte superior da memória, como mostrado em (b), ou os drivers de dispositivo podem estar na parte superior da memória em ROM e o resto do sistema em RAM embaixo, como mostrado em (c).

O primeiro modelo era usado antigamente em computadores de grande porte e minicomputadores e raramente foi utilizado depois disso. O segundo modelo é usado em alguns computadores portáteis e sistemas embarcados. O terceiro modelo foi empregado nos primeiros computadores pessoais (por exemplo, executando o MS-DOS), em que a porção do sistema na ROM é chamada de BIOS (*baste input output system* — sistema básico de E/S). Os modelos (a) e (c) apresentam a desvantagem da possibilidade de que um erro no programa do usuário apague o sistema operacional, possivelmente com resultados desastrosos (como a adulteração do disco).

Quando o sistema é organizado dessa forma, geralmente apenas um processo pode ser executado por vez. Assim que o usuário digita um comando, o sistema operacional copia o programa solicitado do disco para a memória e o executa. Quando o processo termina, o sistema operacional exibe um *prompt* e espera por um novo comando. Quando recebe o novo comando, carrega um novo programa na memória, sobrescrevendo o primeiro.

Um modo de obter algum grau de paralelismo em um sistema sem abstração de memória é programar com múltiplos threads. Uma vez que se suponha que todos os threads em um processo consideram a mesma imagem da memória, o fato de eles serem forçados não é um problema.

Embora essa ideia funcione, sua utilidade é limitada, porque as pessoas normalmente desejam que programas não relacionados sejam executados ao mesmo tempo, algo que a abstração de threads não proporciona. Além do mais, é improvável que sistemas tão primitivos, que não comportam abstração de memória, sejam capazes de fornecer abstração de threads.

9.2 Executando múltiplos programas sem abstração de memória

Mesmo sem abstração de memória, entretanto, é possível executar múltiplos programas simultaneamente. O que o sistema operacional deve fazer é salvar o conteúdo completo da memória em um arquivo de disco e, em seguida, introduzir e executar o próximo programa. Contanto que haja apenas um programa por vez na memória, não há conflitos. Esse conceito (troca de processos, swapping) será discutido adiante.

O acréscimo de hardware especial possibilita executar múltiplos programas simultaneamente, mesmo sem troca de processos. Os primeiros modelos do IBM 360 resolveram o problema do seguinte modo: a memória foi dividida em blocos de 2 KB e a cada um foi atribuída uma chave de proteção de 4 bits mantida em registradores especiais dentro da CPU. Uma máquina com uma memória de 1 MB precisava de apenas 512 desses registradores de 4 bits para um total de 256 bytes de armazenamento de chaves. A PSW (*program status word* palavra de status do programa) também continha uma chave de 4 bits.

O hardware 360 interrompia qualquer tentativa de um processo em execução de acessar a memória com um código de proteção diferente da chave PSW. Visto que apenas o sistema operacional poderia alterar as chaves de proteção, impedia-se que os processos do usuário interferissem no funcionamento mútuo e do próprio sistema operacional.

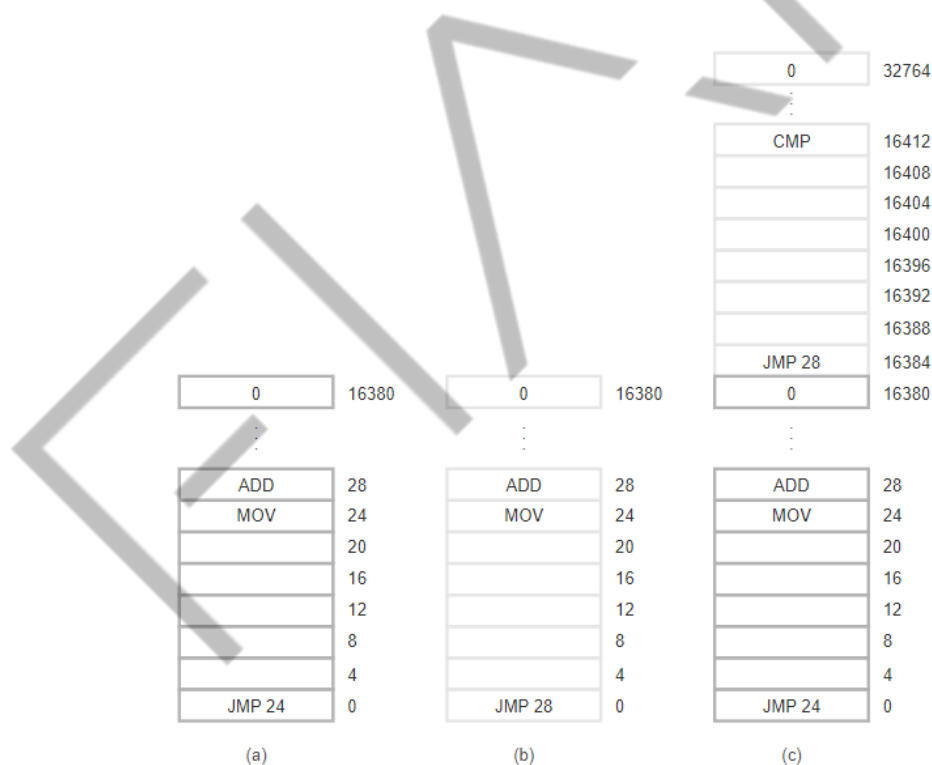


Figura 21 – Ilustração do problema de realocação
Fonte: TANENBAUM (2009)

Entretanto, essa solução tem uma desvantagem importante, ilustrada na Figura “Ilustração do problema de realocação”. Temos aqui dois programas, cada um com 16 KB, como mostrado em (a) e (b).

A primeira é sombreada para indicar que tem uma chave de memória diferente da segunda. O primeiro programa inicializa saltando para o endereço 24, que contém uma instrução MOV. O segundo programa inicializa saltando para o endereço 28, que contém uma instrução CMP. As instruções irrelevantes para essa discussão não são mostradas. Quando os dois programas são carregados sucessivamente na memória, começando no endereço 0, temos a situação em (c). Nesse exemplo, supomos que o sistema operacional está na região alta memória e, dessa forma, não é mostrado.

Depois de serem carregados, os programas podem ser executados. Visto que têm chaves de memória diferentes, um não pode danificar o outro. Mas o problema é de natureza diferente. Quando o primeiro programa é inicializado, executa a instrução JMP 24, que salta para a instrução como esperado. Esse programa funciona normalmente.

No entanto, após ter executado o primeiro programa por tempo suficiente, o sistema operacional pode decidir executar o segundo programa, que foi carregado acima do primeiro no endereço 16.3810. A primeira instrução executada é JMP 28, que salta para a instrução ADD do primeiro programa, em vez de saltar para a instrução esperada, a CMP. O programa provavelmente quebrará muito antes de 1 segundo.

O problema central, nesse caso, é que ambos os programas referenciam a memória física absoluta e não é isso o que queremos. Desejamos que cada programa referencie um conjunto privado de endereços que seja local para o programa.

Explicaremos com poucas palavras como isso pode ser realizado. A solução temporária encontrada pelo IBM 360 foi modificar o segundo programa dinamicamente, à medida que o carregava na memória, usando uma técnica conhecida como realocação estática, que funciona da seguinte forma: quando um programa era carregado no endereço 16.384, a constante 16.384 era adicionada a todos os endereços de programa durante o processo de carregamento.

Embora esse mecanismo funcione se executado corretamente, não é uma solução muito comum e deixa mais lento o carregamento. Além do mais, requer informações adicionais em todos os programas executáveis para indicar quais palavras contêm ou não endereços (relocalizáveis). Afinal, o '28' na Figura “Ilustração do problema de realocação” (b) deve ser relocada, mas uma instrução como:

MOV REGISTER1.28

O que move o número 28 para REGISTER1 não deve ser relocada. O carregador precisa de algum modo dizer o que é um endereço e o que é uma constante.

Por fim, a história tende a se repetir no ramo da computação. Embora o endereçamento direto de memória física seja uma memória distante (perdoem-me) em computadores de grande porte, minicomputadores, computadores de mesa e notebooks, a ausência de abstração de memória ainda é comum em sistemas embarcados e de cartões inteligentes (*smart cards*).

Dispositivos como rádios, máquinas de lavar e fornos de micro-ondas atualmente têm muitos softwares (na ROM) e, na maioria dos casos, o software se endereça à memória absoluta. Isso funciona porque todos os programas são conhecidos antecipadamente e os usuários não são livres para executar seu próprio software na torradeira.

Sistemas embarcados sofisticados (como telefones celulares) têm sistemas operacionais elaborados, ao passo que sistemas mais simples não os têm. Em alguns casos, há um sistema operacional, mas é apenas uma biblioteca vinculada com o programa aplicativo que fornece chamadas de sistema para executar E/S e outras tarefas comuns. O popular sistema operacional e-cos é um exemplo comum de um sistema operacional como biblioteca.

9.3 Noção de endereçamento

Para que múltiplas aplicações estejam na memória simultaneamente, sem interferência mútua, dois problemas devem ser resolvidos: proteção e realocação. Examinamos uma solução primitiva para o primeiro, usada no IBM 360: rotular blocos da memória com uma chave de proteção e comparar a chave do processo em execução com a de cada palavra da memória recuperada. Entretanto, essa abordagem por si só não resolve o último problema, embora ele possa ser resolvido realocando os programas quando são carregados, mas essa é uma solução lenta e complicada.

Uma solução melhor é inventar uma abstração para a memória: o espaço de endereçamento. Assim como o conceito de processo cria um tipo de CPU abstrata para executar programas, o espaço de endereçamento cria um tipo de memória abstrata para abrigá-los. Um espaço de endereçamento é o conjunto de endereços que um processo pode usar para endereçar a memória. Cada processo tem seu próprio espaço de endereçamento, independente dos que pertencem a outros processos (exceto em algumas circunstâncias especiais, em que os processos desejam compartilhar seus espaços de endereçamento).

O conceito de espaço de endereçamento é muito geral e ocorre em muitos contextos. Considere os números de telefone. Nos Estados Unidos e em muitos outros países, um número de telefone local é normalmente um número de 7 dígitos. O espaço de endereçamento para números de telefone varia, desse modo, de 0.000.000 a 9.999.999, embora alguns números, como os começados por 000, não sejam usados.

Com o aumento de telefones celulares, modems e máquinas de fax, esse espaço está se tornando muito pequeno, caso em que mais dígitos têm de ser usados. O espaço de endereçamento para portas de E/S no Pentium varia de 0 a 16383. Endereços IPv4 são números de 32 bits; portanto, seu espaço de endereçamento varia de 0 a $2^{32} - 1$ (novamente com alguns números reservados).

Os espaços de endereçamento não precisam ser numéricos. O conjunto de domínios da Internet xont também é um espaço de endereçamento. Esse espaço de endereçamento consiste de todas as cadeias de comprimento de 2 a 63 caracteres que possam ser feitas usando letras, números e hifens, seguidas por .cont. Agora, você já deve ter uma ideia do que são os espaços de endereçamento. É algo bastante simples.

Dar a cada programa seu próprio espaço de endereçamento, de modo que o endereço 28 em um programa signifique uma localização física diferente do endereço 28 em outro programa, é um pouco mais difícil. Adiante, discutiremos um modo simples, antes bastante comum, mas que caiu em desuso em decorrência da capacidade de colocação de esquemas muito mais complicados (e melhores) em chips de CPUs modernas.

10 MEMÓRIA VIRTUAL

Embora registradores-base e registradores-limite possam ser usados para criar a abstração de espaços de endereçamento, há outro problema a ser resolvido: gerenciar o *bloatware*. O tamanho das memórias está aumentando rapidamente, mas o tamanho dos softwares está aumentando muito mais rápido.

Na década de 1980, muitas universidades executavam um sistema de tempo compartilhado com dezenas de usuários (mais ou menos satisfeitos) simultâneos em um VAX 4 MB. Desde o Microsoft Vista, a Microsoft recomenda pelo menos 512 MB para que um único usuário execute aplicações simples e 1 GB se estiver fazendo algo sério. A tendência à multimídia gera ainda mais demandas sobre a memória. (Nos dias de hoje, considere pelo menos 2GB, certo?)

Como consequência desses desenvolvimentos, há a necessidade de executar programas grandes demais para se encaixarem na memória e, certamente, é necessário haver sistemas que possam dar suporte a múltiplos programas em execução simultânea, cada um dos quais sendo comportado pela memória individualmente, mas que, coletivamente, excedam a memória.

A troca de processos não é uma opção atrativa, visto que um disco SATA típico tem uma taxa de transferência de pico de, no máximo, 100 MB/s, o que significa que leva pelo menos 10 segundos para sair de um programa de 1 GB e outros 10 segundos para inicializar um programa de 1 GB.

10.1 Paginação

A maioria dos sistemas com memória virtual utiliza uma técnica denominada paginação. Em qualquer computador existe um conjunto de endereços de memória que os programas podem gerar ao serem executados. Quando um programa usa uma instrução do tipo:

MOV REG, 1000

Ele deseja copiar o conteúdo do endereço de memória 1000 para o registrador REG (ou o contrário, dependendo do computador). Endereços podem ser gerados com o uso da indexação, de registradores-base, registradores de segmento ou outras técnicas.

Esses endereços gerados pelo programa são denominados endereços virtuais e constituem o espaço de endereçamento virtual. Em computadores sem memória virtual, o endereço virtual é idêntico ao endereço físico e, assim, para ler ou escrever uma posição de memória, ele é colocado diretamente no barramento da memória.

Quando a memória virtual é usada, o endereço virtual não é colocado diretamente no barramento da memória. Em vez disso, ele vai a uma MMU (*memory management unit* — unidade de gerenciamento de memória), que mapeia endereços virtuais em endereços físicos, como ilustrado na Figura “A posição e a função da MMU”.

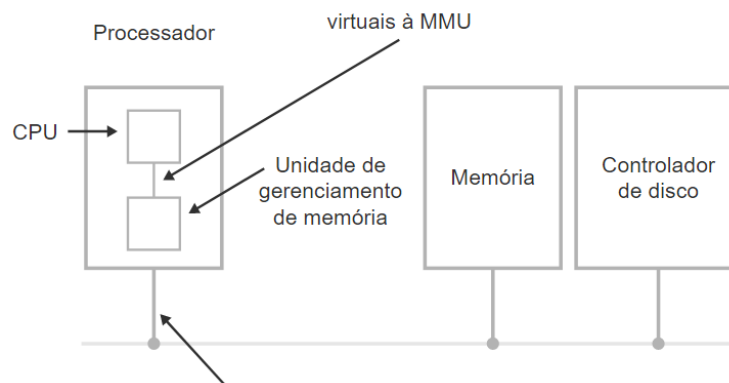


Figura 22 – A posição e a função da MMU
Fonte: TANENBAUM (2009)

Aqui, a MMU é mostrada como parte do chip da CPU (processador), porque isso é comum atualmente. Contudo, em termos lógicos, poderia ser um chip separado, como ocorria no passado.

10.2 Tabela de páginas

No caso mais simples, o mapeamento dos endereços virtuais em endereços físicos pode ser resumido da seguinte forma: o endereço virtual é dividido em um número de página virtual (bits mais significativos) e um deslocamento (bits menos significativos).

Por exemplo, com um endereço de 16 bits e um tamanho de página de 4 KB, os 4 bits superiores poderiam especificar uma dentre as 16 páginas virtuais e os 12 bits inferiores especificariam, então, o deslocamento do byte (de 0 a 4095) dentro da página selecionada. Contudo, também é possível uma divisão com 3 ou 5 bits ou algum outro número de bits para endereçamento de página. Diferentes divisões implicam diferentes tamanhos de páginas.

O número da página virtual é usado como índice dentro da tabela de páginas para encontrar a entrada dessa tabela associada à página virtual em questão. A partir dessa entrada, chega-se ao número da moldura de página física correspondente (caso ela já exista). O número da moldura de página física é concatenado aos bits do deslocamento, substituindo, assim, o número da página virtual pelo da moldura de página física, para formar o endereço físico que será enviado à memória.

Desse modo, o objetivo da tabela de páginas é mapear páginas virtuais em molduras de página física. Matematicamente falando, a tabela de páginas é uma função que usa o número da página virtual como argumento e tem o número da moldura de página física correspondente como resultado. Usando o resultado dessa função, o campo que endereça a página virtual em um endereço virtual pode ser substituído pelo campo que endereça a moldura de uma página física, formando, assim, um endereço da memória física.

10.3 Estrutura de uma entrada da tabela de páginas

Passemos, então, da análise da estrutura das tabelas de páginas como um todo para o detalhamento de uma única entrada da tabela de páginas. O esquema exato de uma entrada é altamente dependente da máquina, mas o tipo de informação presente é aproximadamente o mesmo de máquina para máquina.

Na Figura “Entrada típica de uma tabela de páginas”, mostramos um exemplo de uma entrada da tabela de páginas. O tamanho varia de um computador para o outro, mas 32 bits são um tamanho comum.

O campo mais importante é o Número da moldura de página. Afinal de contas, o objetivo do mapeamento de páginas é localizar esse valor. Próximo a ele, temos o bit Presente/ausente. Se esse bit for 1, a entrada será válida e poderá ser usada. Se ele for 0, a página virtual à qual a entrada pertence não estará presente na memória no referido instante. O acesso à entrada da tabela de páginas com esse bit em 0 causa uma falta de página.

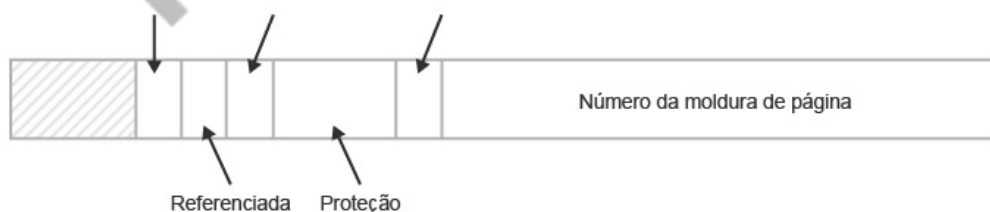


Figura 23 – Entrada típica de uma tabela de páginas
Fonte: TANENBAUM (2009)

Os bits de proteção dizem quais tipos de acesso são permitidos à página. Em sua configuração mais simples, esse campo contém 1 bit, com 0 para leitura/escrita e 1 somente para leitura. Uma forma mais sofisticada tem 3 bits, 1 bit para habilitar/desabilitar cada uma das operações básicas na página: leitura, escrita e execução.

Os *bits* Modificada e Referenciada controlam o uso da página. Ao escrever na página, o hardware automaticamente faz o bit Modificada igual a 1 na entrada correspondente da tabela de páginas. Esse bit é importante quando o sistema operacional decide reivindicar uma moldura de página.

Se a página física dentro dessa moldura foi modificada (isto é, ficou 'suja'), ela também deve ser atualizada no disco. Do contrário (isto é, se continua 'limpa'), ela pode ser simplesmente abandonada, visto que a imagem em disco continua válida. Esse bit é chamado algumas vezes de bit sujo, já que reflete o estado da página.

Ao *bit* Referenciada, é atribuído o valor 1 quando a página física é referenciada, para leitura ou escrita. Esse bit ajuda o sistema operacional a escolher uma página a ser substituída quando ocorre uma falta de página. As páginas físicas que não estão sendo utilizadas são melhores candidatas do que as que estão, e esse bit desempenha um papel importante em vários dos algoritmos de substituição de páginas que estudaremos posteriormente.

Por fim, o último bit permite que o mecanismo de *cache* seja desabilitado para a página em questão. Essa propriedade é mais importante para as páginas que mapeiam em registradores de dispositivos do que na memória. Se o sistema operacional estiver preso em um laço (*loop*) à espera da resposta de algum dispositivo de E/S, é essencial que o hardware mantenha a busca da palavra a partir do dispositivo, e não a partir de uma cópia antiga da cache.

Com esse bit, o uso da cache pode ser desabilitado. Máquinas com espaços de endereçamento separados para E/S e que não usem E/S mapeada em memória não precisam desse bit.

Note que o endereço em disco empregado para armazenar a página quando ela não está presente na memória não faz parte da tabela de páginas. A justificativa para isso é simples: a tabela de páginas contém somente as informações necessárias ao hardware para traduzir um endereço virtual em um endereço físico.

As informações de que o sistema operacional precisa para tratar as faltas de página são mantidas em tabelas em software dentro do sistema operacional. O hardware não necessita dessas informações.

Antes de passar a mais problemas de implementação, convém enfatizar novamente que o que a memória virtual faz, fundamentalmente, é criar uma nova abstração — o espaço de endereçamento o qual é uma abstração da memória física, assim como um processo é uma abstração do processador físico (CPU).

A memória virtual pode ser implementada dividindo o espaço de endereçamento virtual em páginas e mapeando cada uma delas em alguma moldura de página da memória física ou não as mapeando (temporariamente). Dessa forma, este capítulo é basicamente sobre uma abstração criada pelo sistema operacional e sobre como essa abstração é gerenciada.

CONCLUSÃO

Esperamos que este capítulo tenha esclarecido alguns componentes importantes de um sistema operacional. Tais conhecimentos são imprescindíveis na defesa de ambientes e, posteriormente, na análise de *malwares*.

REFERÊNCIAS

SILBERCHATZ, A.; GALVIN, P.B.; GAGNE. **Sistemas Operacionais**. 2ª. ed. São Paulo: Editora Campus, 2000.

TANENBAUM, A. S. **Sistemas Operacionais Modernos**. 3ª. ed. São Paulo: Editora Person, 2009.

EMEND